

1. INTRODUCCIÓN A C# Y .NET

Índice

1. INTRODUCCIÓN A .NET	3
1.1. MODELOS DE APLICACIÓN.....	3
1.2. COMPONENTES DE LA ARQUITECTURA .NET.....	4
1.2.1. Implementaciones.....	4
1.2.2. .NET Standard.....	5
1.2.3. Entornos de Tiempo de Ejecución.....	6
1.2.4. Infraestructura Común	6
1.2.5. Herramientas y Editores .NET.....	7
3. INTRODUCCIÓN A VISUAL STUDIO .NET	7
4. EL ENTORNO DE DESARROLLO INTEGRADO.....	7
4.1. VENTANA DE INICIO.....	8
4.1.1. Comenzar un Nuevo Proyecto	8
4.2. EL EXPLORADOR DE SOLUCIONES.....	11
4.3. MENÚS.....	11
4.4. LISTA DE ERRORES.....	12
4.5. RESULTADOS.....	12
4.6. CUADRO DE BÚSQUEDA	13
4.7. MENÚ ENVIAR COMENTARIOS	14
4.8. OTRAS OPCIONES.....	14
5. EL LENGUAJE C#.....	15
5.1. UNA INTRODUCCIÓN AL LENGUAJE C#	15
6. SINTAXIS DE C#	16
6.1. CASE SENSITIVE	16
6.2. COMENTARIOS.....	16
6.3. CONSTANTES	16
6.4. VARIABLES	17
6.4.1. Nombre de una Variable	17
6.4.2. Declaración de Variables	17
6.4.2.1. DECLARACIÓN EXPLÍCITA.....	17
6.4.2.2. DECLARACIÓN IMPLÍCITA	18
6.4.3. Tipos de Variables	20
6.5. OPERADORES	22
6.6. ESTRUCTURAS DE CONTROL.....	23
6.6.1. Estructuras Condicionales.....	23
6.6.1.1. LA INSTRUCCIÓN IF	23
6.6.1.2. LA INSTRUCCIÓN SWITCH	25
6.6.2. Bucles.....	26
6.6.2.1. LA INSTRUCCIÓN WHILE.....	26
6.6.2.2. LA INSTRUCCIÓN DO...WHILE	27
6.6.2.3. LA INSTRUCCIÓN FOR.....	28

6.6.2.4. LA INSTRUCCIÓN FOREACH	28
6.6.2.5. INSTRUCCIONES DE SALTO	29
6.7. CASTING O CONVERSIÓN DE TIPOS.....	32
6.8. MATRICES (ARRAYS)	33
6.8.1. Matrices Unidimensionales.....	34
6.8.2. Matrices Multidimensionales.....	35
6.8.3. Matrices Escalonadas	37
6.9. MÉTODOS.....	40
6.9.1. Estructura de un Método.....	40
6.9.2. Tipos de Métodos en C#	42
6.9.3. Métodos de Expresión Corporal.....	42
6.9.3. Métodos que Devuelven Varios Valores	43
6.9.3.1. PASO DE PARÁMETROS POR REFERENCIA	43
6.9.3.2. TUPLAS.....	44
6.9.4. Métodos Anidados o Funciones Locales	45
6.9.5. Más opciones para el Paso de Parámetros	46
6.9.5.1. PARÁMETROS OPCIONALES	46
6.9.5.2. ARGUMENTOS CON NOMBRE.....	47
6.10. MANIPULACIÓN DE CADENAS DE CARACTERES	47
6.10.1. Elementos de Formato	48
6.10.2. Concatenación de cadenas.....	50
6.10.3. Clase StringBuilder.....	50
6.10.2. Propiedades y Métodos de la clase String	52
6.10.2.1. CORRECCIÓN DE CADENAS	52
6.10.2.2. ANÁLISIS DE CADENAS	53
6.10.3. Expresiones Regulares	57
6.10.3.1. ISMATCH	58
6.11. COLECCIONES.....	59
6.11.1. Lists.....	60
6.11.1.1. CUÁNDO USAR LISTS Y ARRAYS	63
6.11.2. HashTables	64

1. INTRODUCCIÓN A .NET

.NET es una plataforma de aplicaciones para la creación y ejecución de servicios web y aplicaciones de Internet. Para ello permite usar una serie de *lenguajes, implementaciones, herramientas y bibliotecas*. Hoy en día es la plataforma de desarrollo de software más usada además de Java.

.NET es una colección de diferentes plataformas de software de Microsoft. El framework original fue desarrollado como una competencia directa a la plataforma Java. Fue anunciado oficialmente por primera vez en el año 2000.

Hasta aproximadamente el 2003, el término .NET sirvió a Microsoft como término de marketing y como palabra de moda para productos nuevos, pero muy diferentes, como *sistemas operativos, servidores y software de oficina*. Más tarde, el término se concentró en el desarrollo de software.

Inicialmente no tuvo mucho éxito, pero el entorno .NET ha cambiado significativamente a lo largo de los años y ha ganado en importancia. Hoy en día, el framework .NET se ha vuelto indispensable.

1.1. MODELOS DE APLICACIÓN

Con .NET podemos crear muchos tipos de aplicación, algunos son multiplataforma, y algunos para un sistema operativo en específico, entre ellos tenemos: *Web, Móvil, Aplicaciones de Escritorio, Juegos, IoT (Internet de las Cosas¹), Aplicaciones ML (Sistemas de Inteligencia Artificial²)*.

Algunas de estas aplicaciones son multiplataforma y otras son específicas para un sistema operativo.



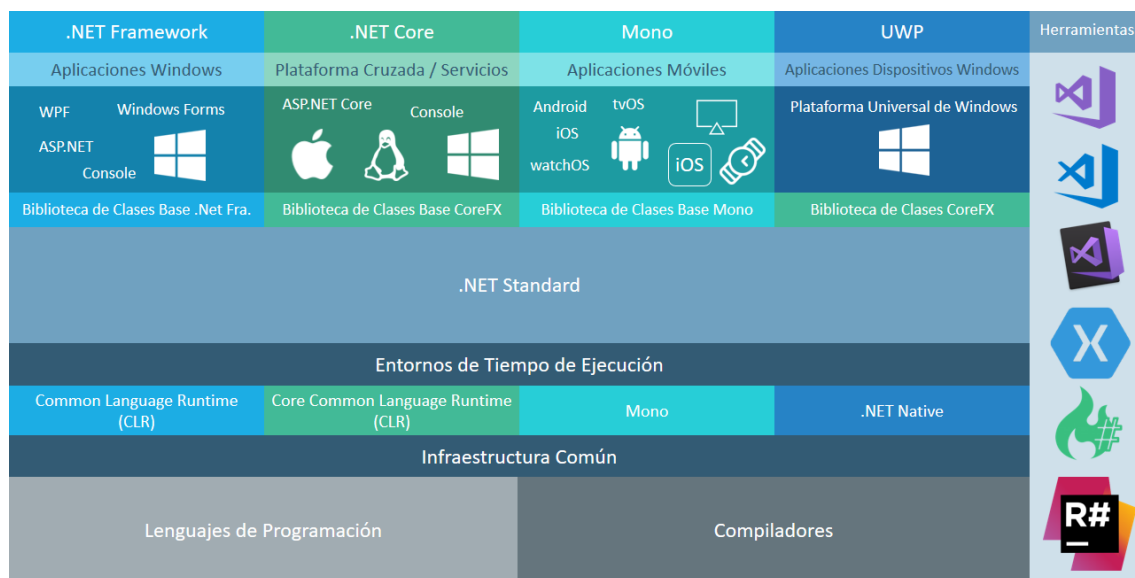
¹ IoT (Internet of Things): engloba todos los dispositivos que se pueden conectar a internet como smartTV, impresora, frigorífico, relojes, etc.

² Aplicaciones ML (Machine Learning): se trata de sistemas de inteligencia artificial como Cortana de Microsoft o Siri de iOS.

1.2. COMPONENTES DE LA ARQUITECTURA .NET

Los componentes de la arquitectura .NET juegan un papel importante en el desarrollo de aplicaciones. Son los siguientes:

- ❖ Las implementaciones.
- ❖ La API³ .NET Standard.
- ❖ Los entornos de tiempo de ejecución.
- ❖ Los Lenguajes de programación y sus compiladores (Tiempo de Compilación).
- ❖ Las herramientas.



1.2.1. Implementaciones

- ❖ **.NET Framework:** es la plataforma tradicional .NET. Es monolítico (necesita instalar toda su base o no puede usarse) y permite la creación de aplicaciones de consola, escritorio, web y móviles. Estas aplicaciones solo se ejecutan en Windows.
- ❖ **.NET Core:** es una nueva generación de tecnologías .NET que se separó por primera vez del .NET Framework en 2015. A diferencia de su versión

³ API es el acrónimo utilizado para *Application Programming Interface*, o en español, Interfaz de Programación de Aplicaciones. Básicamente es una interfaz que permite la comunicación entre dos sistemas o plataformas diferentes, lo que permite agregar diversas funciones a sitios web y aplicaciones.

tradicional, .NET Core está basado y desarrollado con los siguientes objetivos en mente:

- *Más ligero y más modular*: esto permite que nuestra aplicación proporcione exclusivamente lo que necesitamos, sin la necesidad de instalar el conjunto completo de .NET.
- *Multiplataforma*: las aplicaciones que creamos funcionarán en Windows, Linux y MacOS.
- *Alto rendimiento*: .NET Core tiene un rendimiento más alto que la versión tradicional, lo cual es muy importante para los entornos de nube.

❖ **Xamarin/Mono**: es una implementación de .NET que se usa principalmente para las aplicaciones en *Android*, *MacOS*, *iOS*, *tvOS (Apple TV)* y *watchOS (Apple Watch)*.

❖ **UWP**: es la plataforma universal de Windows. Se usa para compilar aplicaciones Windows modernas y táctiles y así poder unificar los diferentes tipos de dispositivos de destino, incluidos equipos, tabletas, teléfonos e incluso la consola Xbox. Aunque algunos desarrolladores la sitúan dentro de la plataforma .NET Core al compartir algunas bibliotecas de este, a Microsoft le gusta que se la considere una implementación más.

1.2.2. .NET Standard

Antes de dar una definición de qué es .NET Standard, tenemos que saber qué es una ***biblioteca de clases portable***, también conocida como **PCL**.

Una **PCL** es una biblioteca que permite compartir el código entre varios proyectos específicos de la plataforma tanto para *IOS*, *Android*, *Windows* o *Windows Phone*.

Las PCL eran una buena solución. No obstante, presentaban muchas desventajas de compatibilidad entre implementaciones. Así por ejemplo, si nuestra clase usaba una API que era de la biblioteca de clases base de *.Net Framework* esta clase no podría ser usada en *.Net Core*, sino que tendríamos que reemplazarla con una equivalente en *.Net Core*; es decir, la misma API no era compatible con implementaciones diferentes. Para solucionar eso, se creó **.Net Standard**, que es una *fusión de las bibliotecas de clases base y las bibliotecas de clases portable*, eso quiere decir que es común para todas las implementaciones de .NET, es decir, *.Net Framework*, *.Net Core*, *Mono* y *UWP*.

Pero en 2017 las PCL quedaron obsoletas y se borraron del sistema, al igual que las bibliotecas base de cada implementación, creándose nuevas bibliotecas en la

API .Net Standard comunes a todas las implementaciones. Aun así, siguen existiendo APIs específicas para cada sistema operativo.

1.2.3. Entornos de Tiempo de Ejecución

El *Entorno de Tiempo de Ejecución* es donde se ejecuta el programa administrado. Estos son los entornos de ejecución .NET según la implementación utilizada:

- ❖ *Common Language Runtime (CLR)* para .NET Framework
- ❖ *Core Common Language Runtime (CoreCLR)* para .NET Core
- ❖ *.NET Native* para la Plataforma universal de Windows (UWP)
- ❖ El entorno de ejecución *Mono* para las aplicaciones de la implementación Xamarin/Mono.

1.2.4. Infraestructura Común

Es la parte común a todas las implementaciones. Ahí estarían:

- ❖ Los **lenguajes de programación**: se utilizan principalmente tres lenguajes de programación: **C#, F# y Visual Basic (VB)**, adicionalmente está el lenguaje **J# (J Sharp)**, que es un lenguaje transicional para programadores de java y J++, sin embargo dejó de producirse en el año 2008.

Por otra parte, es un error muy común, decir que C++ está dentro de .NET. El lenguaje C++ cuenta con su propio compilador y C++ no puede entrar directamente a la plataforma .NET, ya que hay cosas que .NET no soporta. Por lo tanto Microsoft creo C++/CLI, que es básicamente C++ con extensiones de Microsoft que le permiten escribir código dirigido a .NET **Framework**, pero C++ no se incluye en .NET.

Por lo tanto, los únicos lenguajes de .NET son:



- ❖ El **motor de compilación Ms Build**: usado para compilar los proyectos.

1.2.5. Herramientas y Editores .NET

Finalmente, otro componente son las **Herramientas de Desarrollo** para la creación de aplicaciones web o móviles en los diferentes sistemas operativos mencionados:

- ❖ **NuGet**, es un administrador de paquetes de Microsoft .Net donde podemos descargar diferentes bibliotecas que contienen código compilado como archivos *dll*.
- ❖ **Entornos de desarrollo integrado (IDE)**: *Visual Studio*, *Xamarin Studio*, *Visual Studio para Mac*, *JetBrains Rider*.
- ❖ **Editores de Código**: *Visual Studio Code* y *Plugin OmniSharp*.

3. INTRODUCCIÓN A VISUAL STUDIO .NET

Visual Studio .NET es un entorno de desarrollo integrado, lo cual también es conocido como IDE (*Integrated Development Environment*).

Fue creado por la compañía Microsoft y está disponible para sistemas operativos *Windows*, *Linux* y *MacOS*, y a la vez es compatible con múltiples lenguajes de programación, tales como *C++*, *C#*, *Visual Basic .NET*, *F#*, *Java*, *Python*, *Ruby* y *PHP*, al igual que entornos de desarrollo web, como *ASP.NET*. Fue lanzado en 1997, siendo la última versión disponible la de 2022. Dentro de cada versión hay varias ediciones. La que nosotros vamos a utilizar es **Visual Studio 2022 Community**, ya que su uso está precisamente destinado a ámbitos educativos.

4. EL ENTORNO DE DESARROLLO INTEGRADO

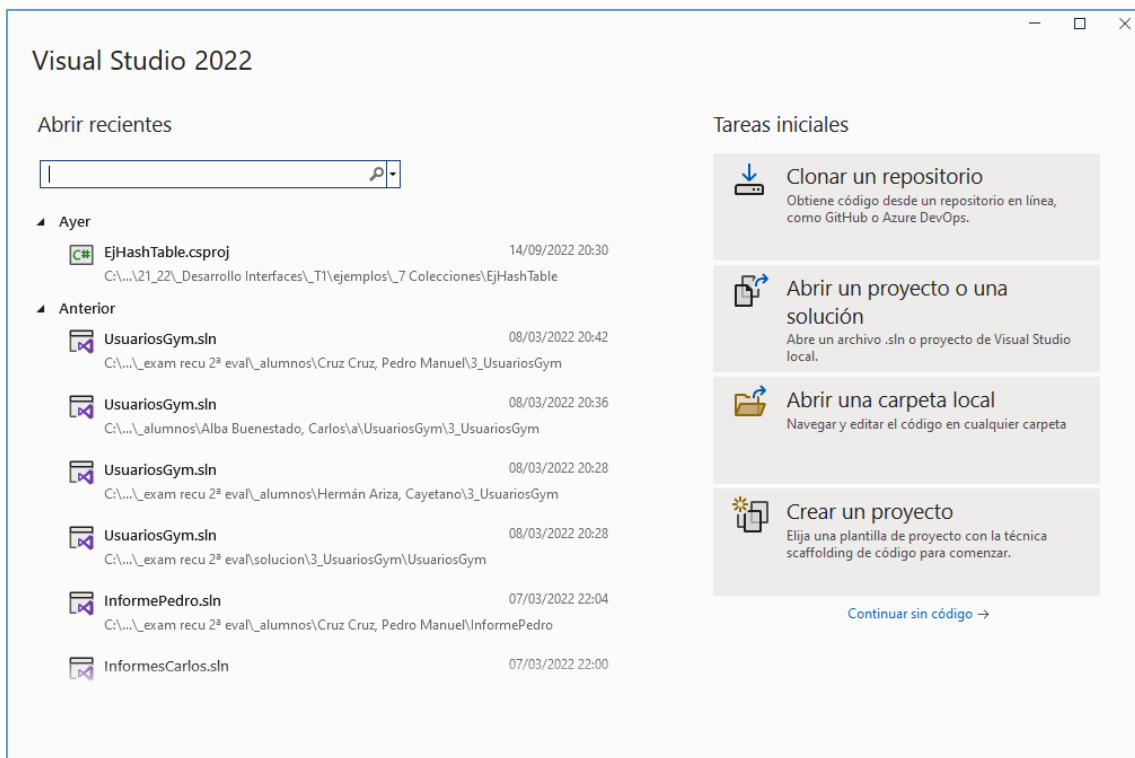
Un *entorno de desarrollo integrado* (IDE) es un programa con numerosas características que se pueden usar para muchos aspectos del desarrollo de software. Todo **IDE** suele comprender un conjunto de herramientas que pueden simplificar bastante el desarrollo de sitios, aplicaciones y servicios web.

Más allá del editor estándar, que cuenta con la opción de *autocompletado*, *indentación de código* y *resaltado de errores*, y el depurador que proporcionan la mayoría de IDEs, Visual Studio incluye compiladores, herramientas de finalización de código, diseñadores gráficos y muchas más características para facilitar el proceso de desarrollo de software.

El paquete del producto ofrece una interfaz visual que permite identificar un programa como servicio web, construir una interfaz de usuario (también soporta interfaces para dispositivos móviles), posee características que permiten la integración de aplicaciones de datos existentes y de depuración.

4.1. VENTANA DE INICIO

Cuando ejecutamos Visual Studio, aparece la ventana mostrada en la figura:



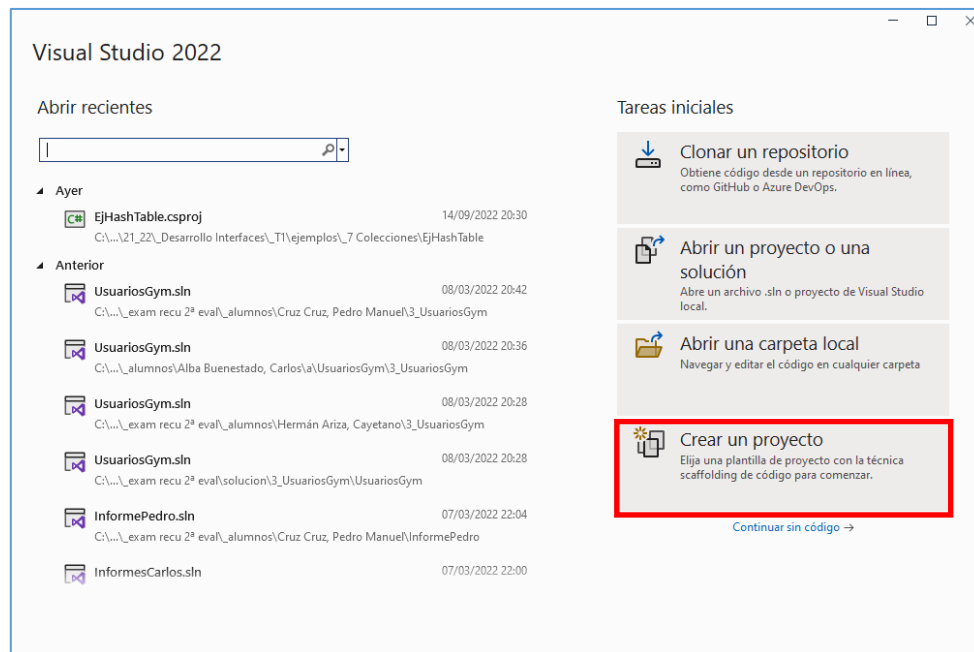
La primera vez que se usa Visual Studio, la lista de *proyectos recientes* está vacía.

La opción *Continuar sin código* simplemente abre el entorno de desarrollo de Visual Studio sin ningún proyecto o código concreto cargado.

4.1.1. Comenzar un Nuevo Proyecto

Con el fin de poder explicar mejor varios elementos del IDE, vamos a crear un nuevo proyecto.

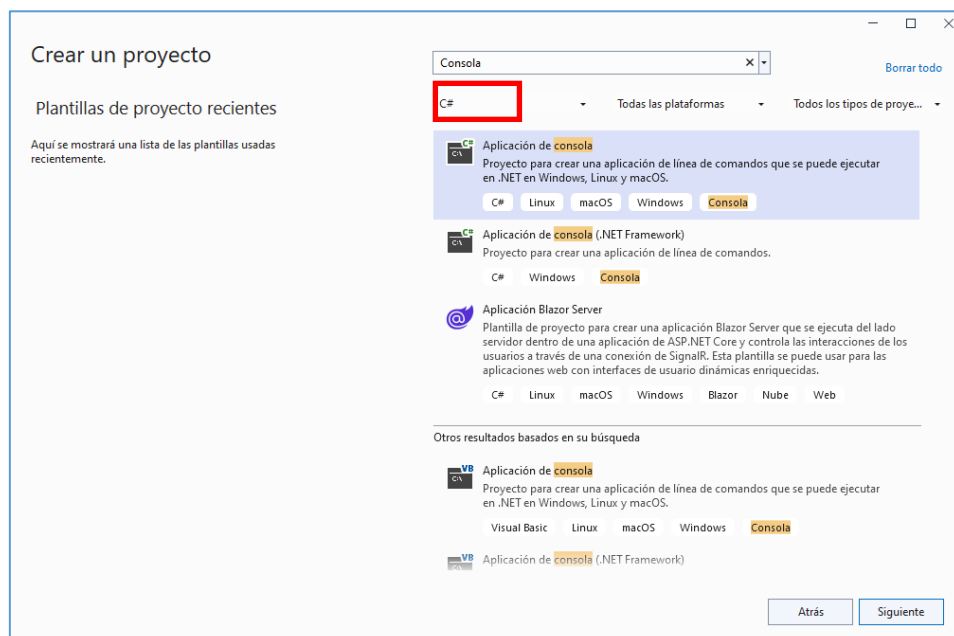
1. En la ventana de inicio, elegimos *Crear un proyecto*.



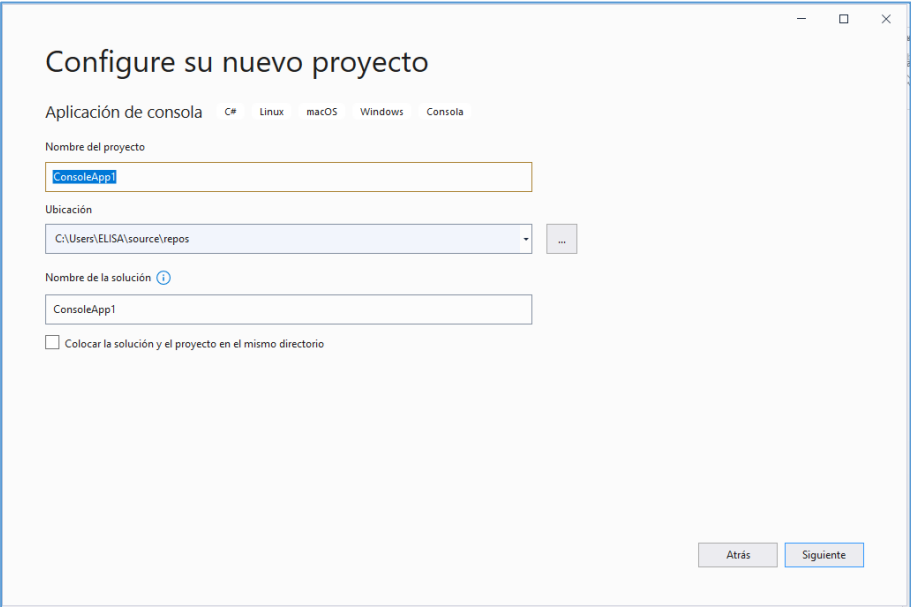
2. La ventana *Crear un proyecto* se abre y muestra varias *plantillas* de proyecto. Una plantilla contiene los archivos básicos y la configuración necesarios para un tipo de proyecto concreto.

Aquí, podemos buscar, filtrar y seleccionar una plantilla de proyecto. También nos muestra una lista de las plantillas de proyecto recientemente usadas.

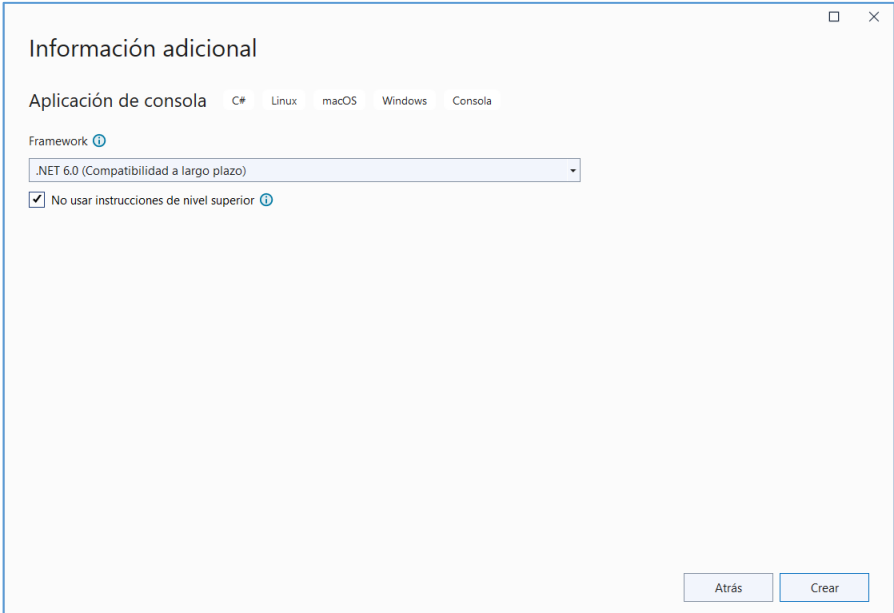
En el cuadro de búsqueda superior, escribimos *consola* para filtrar la lista de tipos de proyecto que incluyen "consola" en el nombre. Para refinar los resultados de la búsqueda, elegimos C# en la lista desplegable *Todos los lenguajes*.



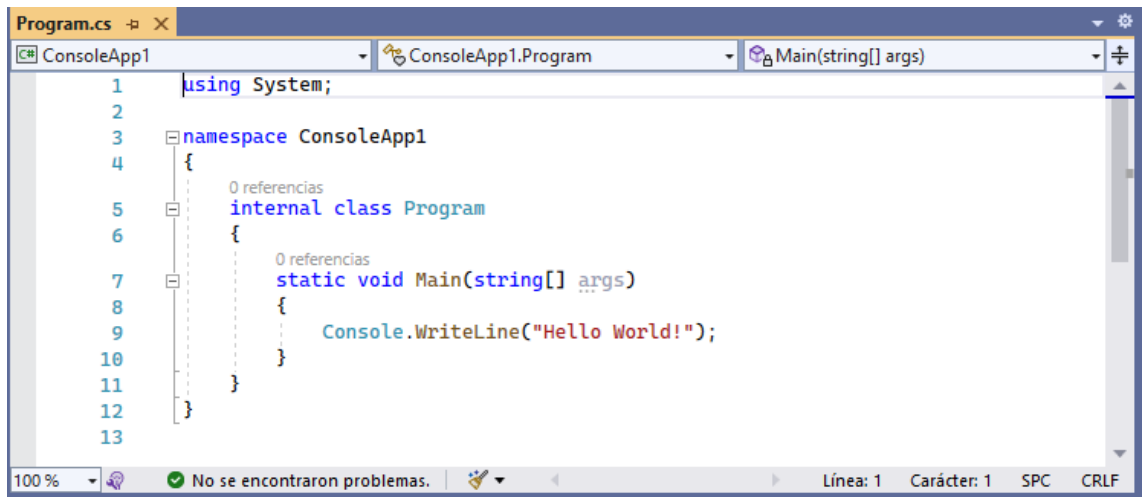
3. Ahora seleccionamos la plantilla *Aplicación de consola* y, luego, elegimos *Siguiente*. Con esta plantilla vamos a crear una aplicación de consola multiplataforma. Es decir, el programa que creemos podrá ejecutarse en Linux, Mac o Windows.
4. En la ventana *Configure su nuevo proyecto*, dejamos el nombre y la ubicación predeterminados del proyecto y hacemos clic en *Siguiente*.



5. En la ventana *información adicional*, seleccionaremos *.NET 6* en el menú desplegable, marcaremos la opción *No usar instrucciones de nivel superior* y haremos clic en *Crear*.

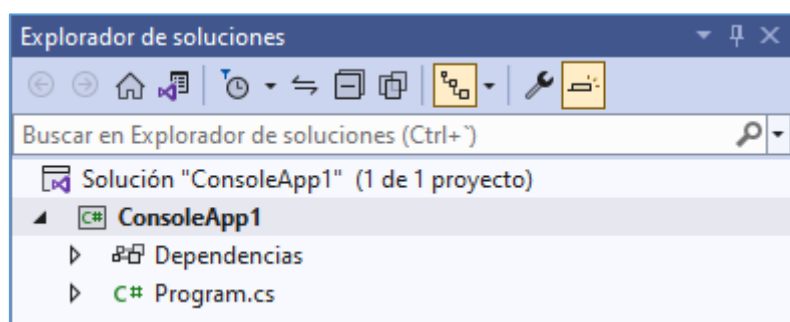


Entonces se habrá creado el proyecto y se abrirá un archivo denominado *Program.cs* en la ventana *Editor*. En esta ventana es donde se muestra el contenido de los archivos y donde se va a realizar la mayor parte del trabajo de codificación en Visual Studio.



4.2. EL EXPLORADOR DE SOLUCIONES

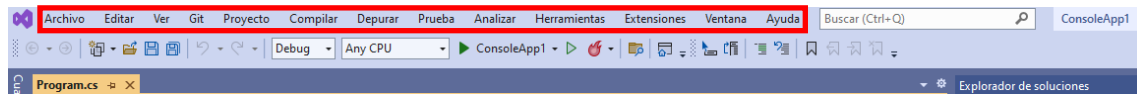
En el *Explorador de soluciones*, que normalmente se encuentra en el lado derecho de Visual Studio, se muestra una representación gráfica de la jerarquía de los archivos y las carpetas del proyecto, la solución o la carpeta de código. En el *Explorador de soluciones* se puede examinar la jerarquía y navegar hasta un archivo.



4.3. MENÚS

La barra de menús de la parte superior de Visual Studio agrupa los comandos en categorías. Por ejemplo, el menú *Proyecto* contiene comandos relacionados con el proyecto en el que está trabajando. En el menú *Herramientas*, se puede personalizar el comportamiento de Visual Studio mediante la selección

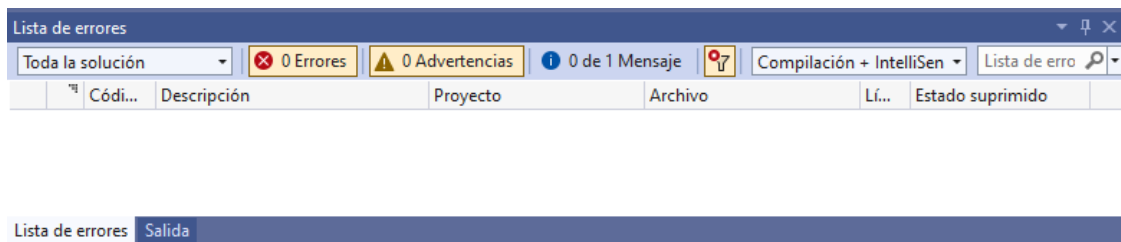
de *Opciones*, o bien agregar características a la instalación mediante la selección de *Obtener herramientas y características*.



4.4. LISTA DE ERRORES

La ventana *Lista de errores* se abre mediante la selección del menú *Ver->Lista de errores*.

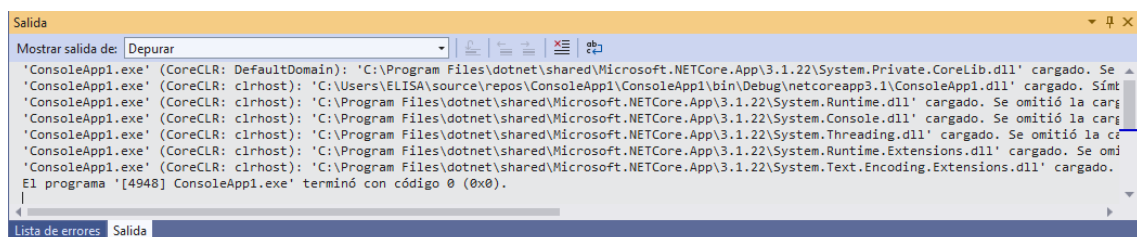
En la *Lista de errores* se muestran los errores, las advertencias y los mensajes relacionados con el estado actual del código. Si se produjera algún error (por ejemplo, la falta de un paréntesis o un punto y coma) en el archivo o en cualquier parte del proyecto, se indicaría aquí.



4.5. RESULTADOS

En la ventana *Resultados* se muestran los mensajes de resultados de la compilación del proyecto y del proveedor de control de código fuente.

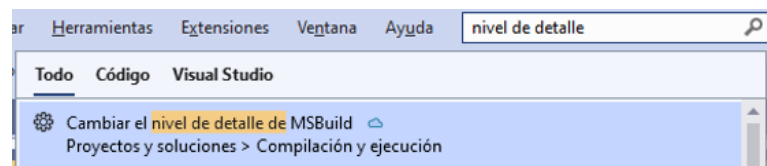
Si compilamos el proyecto de consola creado en apartados anteriores, puesto que está todo correcto, veremos los mensajes que se muestran en la siguiente imagen. Para ello tenemos que acceder a *Compilar->Compilar solución*.



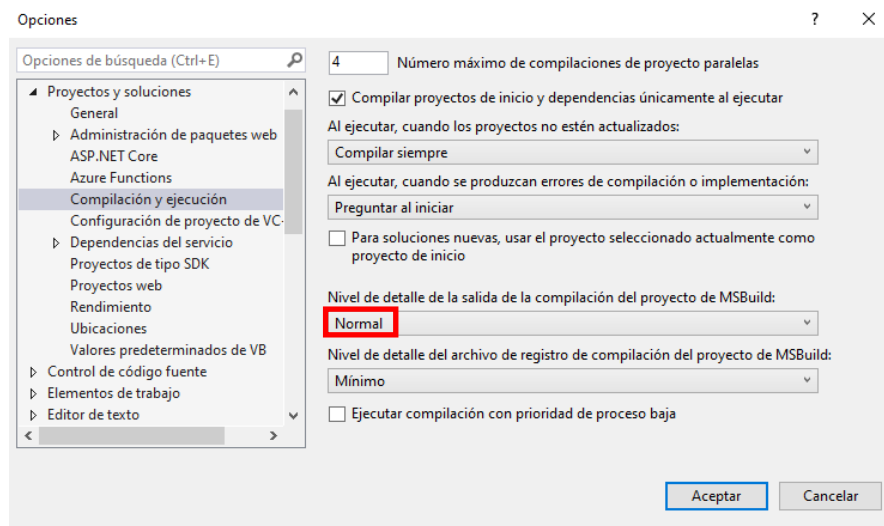
4.6. CUADRO DE BÚSQUEDA

El cuadro de búsqueda es una manera rápida y sencilla de navegar prácticamente por todo en Visual Studio. Se puede especificar algún texto relacionado con lo que se quiere hacer, y se mostrará una lista de opciones que pertenecen al texto. Por ejemplo, supongamos que se quiere aumentar el detalle de los resultados de compilación para mostrar más detalles sobre lo que está haciendo exactamente la compilación. Así es como se haría:

1. Presionamos **Ctrl+Q** para activar el cuadro de búsqueda en la parte superior del IDE.
2. Escribimos *nivel de detalle* en el cuadro de búsqueda. En los resultados que se muestran, elegimos *Cambiar el nivel de detalle de MSBuild*.



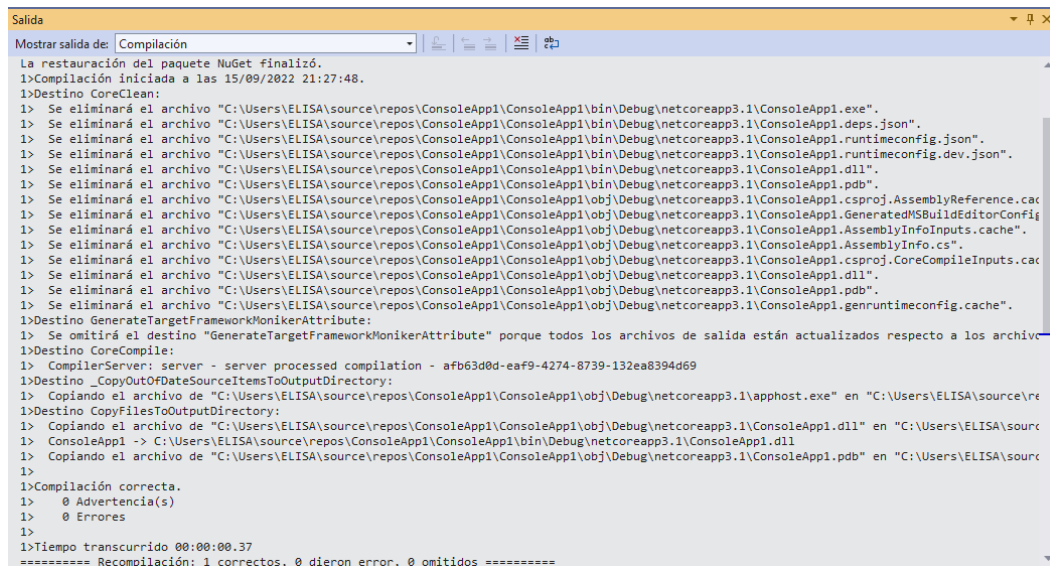
El cuadro de diálogo *Opciones* se abre en la página *Compilación y ejecución*:



3. En *Nivel de detalle de la salida de la compilación del proyecto de MSBuild*, elegimos *Normal* y después hacemos clic en *Aceptar*.

Entonces volveremos a compilar el proyecto haciendo clic con el botón derecho en el proyecto *ConsoleApp1* del *Explorador de soluciones* y elegiremos *Recompilar* en el menú contextual.

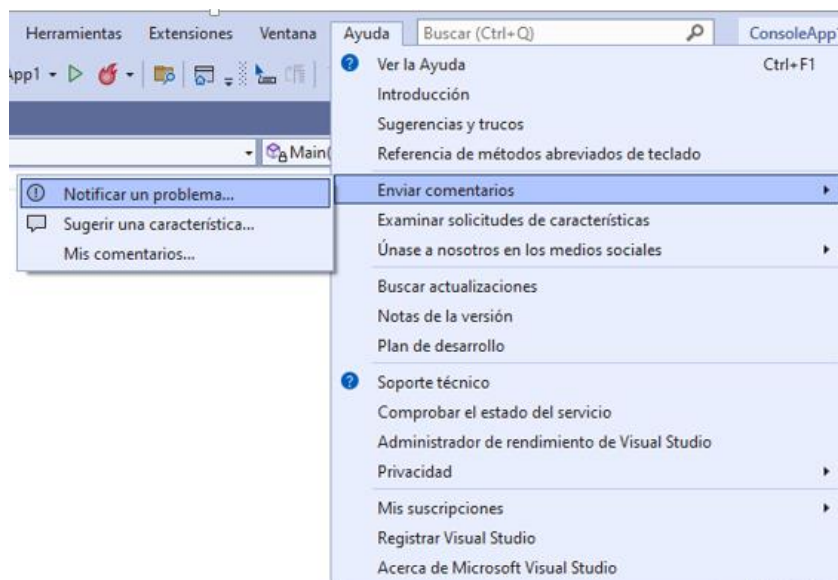
Esta vez, en la ventana *Resultados* se mostrará un registro más detallado del proceso de compilación, incluido dónde fueron copiados los archivos.



```
Salida
Mostrar salida de: Compilación
La restauración del paquete NuGet finalizó.
1>Compilación iniciada a las 15/09/2022 21:27:48.
1>Destino CoreClean:
1> Se eliminará el archivo "C:\Users\ELISA\source\repos\ConsoleApp1\ConsoleApp1\bin\Debug\netcoreapp3.1\ConsoleApp1.exe".
1> Se eliminará el archivo "C:\Users\ELISA\source\repos\ConsoleApp1\ConsoleApp1\bin\Debug\netcoreapp3.1\ConsoleApp1.deps.json".
1> Se eliminará el archivo "C:\Users\ELISA\source\repos\ConsoleApp1\ConsoleApp1\bin\Debug\netcoreapp3.1\ConsoleApp1.runtimeconfig.json".
1> Se eliminará el archivo "C:\Users\ELISA\source\repos\ConsoleApp1\ConsoleApp1\bin\Debug\netcoreapp3.1\ConsoleApp1.dll".
1> Se eliminará el archivo "C:\Users\ELISA\source\repos\ConsoleApp1\ConsoleApp1\bin\Debug\netcoreapp3.1\ConsoleApp1.pdb".
1> Se eliminará el archivo "C:\Users\ELISA\source\repos\ConsoleApp1\ConsoleApp1\obj\Debug\netcoreapp3.1\ConsoleApp1.csproj.AssemblyReference.cache".
1> Se eliminará el archivo "C:\Users\ELISA\source\repos\ConsoleApp1\ConsoleApp1\obj\Debug\netcoreapp3.1\ConsoleApp1.GeneratedMSBuildEditorConfig".
1> Se eliminará el archivo "C:\Users\ELISA\source\repos\ConsoleApp1\ConsoleApp1\obj\Debug\netcoreapp3.1\ConsoleApp1.AssemblyInfoInputs.cache".
1> Se eliminará el archivo "C:\Users\ELISA\source\repos\ConsoleApp1\ConsoleApp1\obj\Debug\netcoreapp3.1\ConsoleApp1.csproj.CoreCompileInputs.cache".
1> Se eliminará el archivo "C:\Users\ELISA\source\repos\ConsoleApp1\ConsoleApp1\obj\Debug\netcoreapp3.1\ConsoleApp1.dll".
1> Se eliminará el archivo "C:\Users\ELISA\source\repos\ConsoleApp1\ConsoleApp1\obj\Debug\netcoreapp3.1\ConsoleApp1.pdb".
1> Se eliminará el archivo "C:\Users\ELISA\source\repos\ConsoleApp1\ConsoleApp1\obj\Debug\netcoreapp3.1\ConsoleApp1.genruntimeconfig.cache".
1>Destino GenerateTargetFrameworkMonikerAttribute:
1> Se omitirá el destino "GenerateTargetFrameworkMonikerAttribute" porque todos los archivos de salida están actualizados respecto a los archivos de entrada.
1>Destino CoreCompile:
1> CompilerServer: server - server processed compilation - afb63d0d-eaf9-4274-8739-132ea8394d69
1>Destino_CopyOutOfDateSourceItemsToOutputDirectory:
1> Copiando el archivo de "C:\Users\ELISA\source\repos\ConsoleApp1\ConsoleApp1\obj\Debug\netcoreapp3.1\apphost.exe" en "C:\Users\ELISA\source\repos\ConsoleApp1\bin\Debug\netcoreapp3.1\apphost.exe".
1>Destino_CopyFilesToOutputDirectory:
1> Copiando el archivo de "C:\Users\ELISA\source\repos\ConsoleApp1\ConsoleApp1\obj\Debug\netcoreapp3.1\ConsoleApp1.dll" en "C:\Users\ELISA\source\repos\ConsoleApp1\bin\Debug\netcoreapp3.1\ConsoleApp1.dll".
1> Copiando el archivo de "C:\Users\ELISA\source\repos\ConsoleApp1\ConsoleApp1\obj\Debug\netcoreapp3.1\ConsoleApp1.pdb" en "C:\Users\ELISA\source\repos\ConsoleApp1\bin\Debug\netcoreapp3.1\ConsoleApp1.pdb".
1>
1>Compilación correcta.
1> 0 Advertencia(s)
1> 0 Errores
1>
1>Tiempo transcurrido 00:00:00.37
===== Recompilación: 1 correctos, 0 dieron error, 0 omitidos =====
```

4.7. MENÚ ENVIAR COMENTARIOS

Si encontramos algún problema mientras estemos usando Visual Studio o si tenemos sugerencias para mejorar el producto, podemos usar el menú *Enviar comentarios*, al que se accede mediante *Ayuda->Enviar comentarios*.



4.8. OTRAS OPCIONES

Solo se han examinado algunas de las características de Visual Studio para familiarizarse con la interfaz de usuario. Para explorarla más a fondo puede consultarse la siguientes url:

[Información general sobre Visual Studio | Microsoft Docs](#)

5. EL LENGUAJE C#

Aunque la plataforma .NET permite programar con múltiples lenguajes, hemos seleccionado C# por ser uno de los más representativos y utilizados de .NET. En este apartado, se presenta una introducción a la sintaxis del lenguaje C# que, tal y como se podrá observar, presenta muchas similitudes con lenguajes como C++ y Java.

5.1. UNA INTRODUCCIÓN AL LENGUAJE C#

La mejor forma de empezar a aprender un lenguaje de programación nuevo suele ser analizando algún ejemplo de código sencillo. Para no romper la tradición, empecemos con el típico *HolaMundo*:

EJEMPLO:

```
using System;

namespace HolaMundo
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hola Mundo");
        }
    }
}
```

Analicemos los elementos del programa anterior:

- ❖ Definición del **namespace**: todas las clases están incluidas dentro de un espacio de nombres concreto, que se indica dentro del código fuente mediante la instrucción *namespace*.
- ❖ Definición de la **clase**: la forma de definir una clase es con la palabra clave *class*, seguida del nombre que queremos dar a la clase, y todos los elementos e instrucciones que pertenecen a la definición de la clase entre llaves. Cabe destacar que en C# todo son clases, con lo que todas las líneas de código deben estar asociadas a alguna clase.
- ❖ El método **main**: punto de entrada a la aplicación, por donde empieza su ejecución. Puede recibir un array llamado *args*, que contiene los parámetros pasados al ejecutable al lanzar su ejecución.

6. SINTAXIS DE C#

La sintaxis de un lenguaje es la definición de las palabras clave, los elementos y las combinaciones válidas en ese lenguaje. A continuación, describimos de forma resumida la sintaxis de C#.

6.1. CASE SENSITIVE

C# es un lenguaje *case sensitive*, es decir, que diferencia entre mayúsculas y minúsculas. Por ejemplo, si escribimos *Class* en lugar de *class*, el compilador dará un error de sintaxis.

6.2. COMENTARIOS

Los comentarios son imprescindibles si pretendemos escribir código de calidad. En C# existen dos tipos de comentarios:

Comentarios	
De una sola línea	// Ejemplo de comentario
De múltiples líneas	/* ... */
	Cualquier carácter entre el símbolo /* y el símbolo */, se considera como parte del comentario, aunque abarque varias líneas.

6.3. CONSTANTES

Una **constante** es una variable cuyo valor no puede ser modificado, es decir, que es de sólo lectura. Se declaran con la palabra clave *const*. Y es bueno usarlas por dos razones:

1. Las constantes no cambian de valor, por lo que podemos estar seguros de que el valor especificado en la declaración de la constante tendrá efecto en el programa entero.
2. Las constantes se procesan más rápido que las variables.

La forma en que se declaran constantes es similar a como se declaran variables con la excepción de que además de facilitar el nombre de la constante, se debe también suministrar un valor:

```
const <tipoConstante> <nombreConstante> = <valorInicial>;
```

EJEMPLOS:

```
const double PI = 3.14159265358979;  
const int NUMERO_ALUMNOS_MAXIMO = 30;
```

6.4. VARIABLES

Una **variable** contiene un valor que puede modificarse a lo largo de la ejecución de la aplicación.

6.4.1. Nombre de una Variable

Las variables deben tener un nombre único llamado **identificador**, los identificadores se forman siguiendo las siguientes reglas:

- Comenzar con una *letra* (de la A-Z o de la a-z).
- Pueden contener *letras, dígitos y guiones bajos*.
- No pueden ser *palabras reservadas*.

Hay que tener en cuenta que C# es sensible al uso de mayúsculas y minúsculas.

Algunas **recomendaciones** respecto a los nombres de las variables son:

- Deben comenzar con una letra.
- Han de distinguir cada nueva palabra escribiendo la primera letra en mayúscula.
Ej. *saldoMovil*
- Que sean lo suficientemente largos como para ser significativos y lo suficientemente cortos como para ser manejables.

6.4.2. Declaración de Variables

6.4.2.1. DECLARACIÓN EXPLÍCITA

Para declarar una variable sólo hay que decir cuál será el nombre que se le dará y cuál será el tipo de datos que podrá almacenar, lo que se hace con la siguiente sintaxis:

```
<tipoVariable> <nombreVariable>;
```

C# también proporciona una sintaxis más sencilla con la que podremos asignar un valor a una variable en el mismo momento que se define. Para ello se la ha de definir usando esta otra notación:

```
<tipoVariable> <nombreVariable> = <valorInicial>;
```

6.4.2.2. DECLARACIÓN IMPLÍCITA

Las *variables locales* pueden declararse sin proporcionar un tipo explícito. La palabra clave **var** indica al compilador que infiera el tipo de la variable a partir de la expresión de la derecha de la instrucción de inicialización.

EJEMPLOS:

```
//i es compilado como un int  
var i = 5;  
  
//s es compilado como un string  
var s = "Hello";
```

Existen ciertas reglas de obligado cumplimiento para usar variables locales implícitamente tipadas:

- ❖ La declaración debe incluir un valor de inicialización. En otras palabras, no será posible usar en una línea `var i;`, puesto que el compilador no podría inferir el tipo en este momento.
- ❖ El valor de inicialización debe ser una expresión evaluable en tiempo de compilación. Expresamente prohibido el valor *null*, es decir, nada de `var n = null;`, puesto que el compilador no sabría de qué tipo se trata.
- ❖ La declaración no puede incluir más de una variable. Una línea como `var i = 1, s = "hola";` generaría un error en compilación.

El uso de **var** ayuda a simplificar el código, pero queda restringido a los siguientes casos:

- 1) Para facilitar la escritura del programa, siempre y cuando no se pierda legibilidad en el código.
- 2) En los bucles **for** y **foreach**.
- 3) Para posibilitar la creación de objetos de tipo **anónimo**.

A continuación vamos a ver ejemplos de cada uno de estos casos.

1) Facilidad a la hora de programar

EJEMPLO 1: Declaración de la variable x

FORMA 1:

```
var x = XmlDateTimeSerializationMode.Local;
```

FORMA 2:

```
XmlDateTimeSerializationMode x = XmlDateTimeSerializationMode.Local;
```

EJEMPLO 2: Declaración de la variable dict1

FORMA 1:

```
var dict1 = new Dictionary<int, List<string>>();
```

FORMA 2:

```
Dictionary<int, List<string>> dict1 = new Dictionary<int, List<string>>();
```

En ambos ejemplos resulta mucho más cómodo declarar la variable utilizando la palabra **var**. De esta forma simplemente hemos indicado que preferimos que sea el compilador el que infiera el tipo y, por así decirlo, haga el trabajo sucio.

El ejemplo siguiente, no obstante, sería un claro ejemplo de cuando no se debe utilizar, dada la dificultad de saber qué tipo devuelve el método llamado y por tanto, la pérdida de legibilidad en el código.

EJEMPLO de mal uso

```
var myVar = SomeMethodWithANameWhichDoesntIndicateTheReturnType();
```

2) En los bucles for y foreach

Otro contexto donde este tipo de variables pueden facilitarnos la vida de forma frecuente es en los bucles **foreach**:

EJEMPLO con FOREACH:

```
// Un ejemplo de bucle...
```

```
var arrayDeChars = "Saludos".ToCharArray(); //matriz de chars
```

```
foreach (var ch in arrayDeChars)
```

```
{
```

```
    Console.Write(ch); //ch es char
```

```
}
```

3) Creación de objetos de tipo anónimo

También es importante la existencia de esta nueva forma de declaración para posibilitar la creación de objetos de tipo **anónimo**, es decir, aquellos cuya especificación de clase se crea en el mismo momento de su definición. Por tanto,

una declaración como la siguiente sería válida:

EJEMPLO para la creación de un tipo ANÓNIMO:

```
var x = new { Nombre = "Juan", Edad = 23 };
```

En temas posteriores se hablará detenidamente de las clases **anónimas**. De momento simplemente nos bastará con saber que una vez compilado el código anterior el resultado será algo parecido al siguiente:

```
class __Anonymous1
{
    private string nombre ;
    private int edad ;
    public string Nombre { get { return nombre ; } }
    public int Edad { get { return edad ; } }
    public __Anonymous1(string nombre, int edad)
    {
        this.nombre = nombre;
        this.edad = edad;
    }
}
...
__Anonymous1 x = new __Anonymous1("Juan", 23);
```

En resumen, la palabra clave **var** permite declarar una variable local (dentro de un método o un bucle) sin indicar explícitamente el tipo - en cambio, el compilador de C # infiere el tipo desde la parte derecha de la declaración. Esto puede ser realmente conveniente en muchas situaciones, pero también puede hacer que nuestro código sea un poco menos legible.

6.4.3. Tipos de Variables

Los tipos de datos básicos son ciertos tipos de datos tan comúnmente utilizados en la escritura de aplicaciones que en C# se ha incluido una sintaxis especial para tratarlos:

EJEMPLO:

```
System.Int32 a = 2;
```

Dado lo frecuente que es el uso de este tipo también se ha predefinido en C# el alias **int** para el mismo, por lo que la definición de variable anterior queda así de compacta:

EJEMPLO:

```
int a = 2;
```

System.Int32 no es el único tipo de dato básico incluido en C#. En el espacio de nombres *System* se han incluido todos estos:

Tipo	Descripción	Bits	Rango de valores	Alias
SByte	Bytes con signo	8	-128 – 127	sbyte
Byte	Bytes sin signo	8	0 – 255	byte
Int16	Enteros cortos con signo	16	[-32.768, 32.767]	short
UInt16	Enteros cortos sin signo	16	[0, 65.535]	ushort
Int32	Enteros normales	32	[-2.147.483.648, 2.147.483.647]	int
UInt32	Enteros normales sin signo	32	[0, 4.294.967.295]	uint
Int64	Enteros largos	64	[-9.223.372.036.854.775.808, 9.223.372.036.854.775.807]	long
UInt64	Enteros largos sin signo	64	[0-18.446.744.073.709.551.615]	ulong
Single	Reales con 7 dígitos de precisión	32	$[1,5 \times 10^{-45} - 3,4 \times 10^{38}]$	float
Double	Reales de 15-16 dígitos de precisión	64	$[5,0 \times 10^{-324} - 1,7 \times 10^{308}]$	double
Decimal	Reales de 28-29 dígitos de precisión	128	$[1,0 \times 10^{-28} - 7,9 \times 10^{28}]$	decimal
Boolean	Valores lógicos	32	true, false	bool
Char	Caracteres Unicode	16	['\u0000', '\uFFFF']	char
String	Cadenas de caracteres	Variable	El permitido por la memoria	string
Object	Cualquier objeto	Variable	Cualquier objeto	object

El valor que por defecto se da a los campos de tipos básicos consiste en poner a cero toda el área de memoria que ocupen. Esto se traduce en que los campos de tipos básicos numéricos se inicializan por defecto con el valor 0, los de tipo **bool** lo hacen con false, los de tipo **char** con '\0', y los de tipo **string** y **object** con *null*.

Para comprobar cuál es el valor por defecto de una variable se le puede asignar el valor por defecto mediante el literal **default**, tal y como se ve en el siguiente código:

EJEMPLO:

```
//Comprobamos el valor por defecto de los tipos más usados
//Declaramos variables
int entero;
float real;
char letra;

//Podemos declarar varias variables del mismo tipo en la misma
línea
string palabra1, palabra2;
bool vf;
object miobjeto;

//Asignamos a las variables el valor por defecto
entero = default;
real = default;
letra = default;
palabra1 = default;
palabra2 = default;
```

```
vf = default;
miobjeto = default;
```

6.5. OPERADORES

La tabla que se muestra a continuación presenta el conjunto de operadores más comunes que soporta C#.

Tipo	Operación	Operador
Aritmético	Cambio de signo	-
	Multiplicación y división	*, /
	Resto de una división entera	%
	Suma y resta	+, -
Cadenas de caracteres	Concatenar o enlazar	+
Incremento / decremento	Incremento y decremento	++, --
Comparación	Igual, distinto, menor, mayor ...	==, !=, <, >, <=, >=
Lógico	Not	!
	And	&&
	Or	
Asignación	Asignar a una variable el resultado de realizar la operación	=, +=, -=, *=, /=, %=

EJEMPLO de operadores aritméticos:

```
int a, b, c;
a = 2;
Console.WriteLine("a=" + a);
b = a; //b=2
Console.WriteLine("b=" + b);
c = b++; //c=2 y b=3
Console.WriteLine("c=" + c + " y b=" + b);

b = a + b * c; //b=2+3*2=8
Console.WriteLine("b=" + b);

//si a>=100 => resultado=b; si no, resultado=c/10
c = a >= 100 ? b : c / 10;
Console.WriteLine("La variable c=" + c); //c=2/10=0
```

```
a = (int)Math.Sqrt(b + c * c); // Raíz de (8 + 0)= Raíz de 8
Console.WriteLine("La raíz de " + b + " + " + c + "* " + c + "= " +
Math.Sqrt(b + c * c));
Console.WriteLine("La parte entera de la raíz es " + a);
Console.ReadLine();
```

EJEMPLO de asignación:

```
int a = 5;
a += 9;
Console.WriteLine(a); // salida: 14

a -= 4;
Console.WriteLine(a); // salida: 10

a *= 2;
Console.WriteLine(a); // salida: 20

a /= 4;
Console.WriteLine(a); // salida: 5

a %= 3;
Console.WriteLine(a); // salida: 2
```

6.6. ESTRUCTURAS DE CONTROL

Todos los programas están hechos con las sentencias mostradas en este apartado. Sin la capacidad de controlar el flujo del programa, los ordenadores serían sólo calculadoras voluminosas. Necesitaremos utilizar las sentencias siguientes para escribir programas que reaccionen ante eventos externos y produzcan los resultados deseados bajo todas las circunstancias.

6.6.1. Estructuras Condicionales

Las estructuras condicionales son instrucciones que permiten ejecutar bloques de instrucciones sólo si se da una determinada *condición*. En los siguientes apartados de este epígrafe se describen cuáles son las instrucciones condicionales disponibles en C#.

6.6.1.1. LA INSTRUCCIÓN IF

La instrucción **if** permite ejecutar ciertas instrucciones sólo si se da una determinada *condición*. Su sintaxis de uso es la sintaxis:

```
if (<condición>)  
{  
    <instruccionesIf>  
}  
else  
{  
    <instruccionesElse>  
}
```

EJEMPLO:

```
int time = 22;  
if (time < 20)  
{  
    Console.WriteLine("Buenos días.");  
}  
else  
{  
    Console.WriteLine("Buenas noches.");  
}
```

También podemos usar la instrucción **else if** cuando la primera condición sea falsa:

```
if (<condición1>)  
{  
    <instruccionesCondicion1Verdadera>  
}  
else if (<condición2>)  
{  
    <instruccionesCondición2Verdadera>  
}  
else  
{  
    <instruccionesElse>  
}
```

EJEMPLO:

```
int time = 22;  
if (time < 12)  
{  
    Console.WriteLine("Buenos días.");  
}  
else if (time < 20)  
{  
    Console.WriteLine("Buenas tardes");  
}  
else  
{  
    Console.WriteLine("Buenas noches");  
}
```

Finalmente, también tenemos la opción siempre de utilizar **if anidados**:

```
if (<condición1>)
{
    <instruccionesCondicion1Verdadera>
}
else
{
    if (<condición2>)
    {
        <instruccionesCondición2Verdadera>
    }
    else
    {
        <instruccionesElse>
    }
}
```

EJEMPLO:

```
int time = 22;
if (time < 12)
{
    Console.WriteLine("Buenos días.");
}
else
{
    if (time < 20)
    {
        Console.WriteLine("Buenas tardes");
    }
    else
    {
        Console.WriteLine("Buenas noches");
    }
}
```

6.6.1.2. LA INSTRUCCIÓN SWITCH

La instrucción **switch** permite ejecutar unos u otros bloques de instrucciones según el valor de una cierta expresión. Su estructura es:

```
switch (<expresión>)
{
    case <valor1>: <bloque1>
                  break;
    case <valor2>: <bloque2>
                  break;
    ...
    default:      <bloqueDefault>
                  break;
}
```

Vamos a ver cómo quedaría el ejemplo visto con los **if anidados** utilizando un **switch**:

EJEMPLO:

```
Console.WriteLine("¿Qué hora es?");
int time = Convert.ToInt32(Console.ReadLine());
switch( time)
{
    case 6:
    case 7:
    case 8:
    case 9:
    case 10:
    case 11:      Console.WriteLine("Buenos días.");
                  break;
    case 12:
    case 13:
    case 14:
    case 15:
    case 16:
    case 17:
    case 18:
    case 19:      Console.WriteLine("Buenos tardes.");
                  break;
    default:      Console.WriteLine("Buenas noches");
                  break;
}
```

6.6.2. Bucles

Los bucles son una parte muy importante de cualquier lenguaje de programación, porque muchas tareas se basan en la repetición una y otra vez de operaciones triviales. C# dispone de los tipos siguientes de bucles:

- ❖ **while**
- ❖ **do...while**
- ❖ **for**
- ❖ **foreach**

6.6.2.1. LA INSTRUCCIÓN WHILE

La instrucción **while** permite ejecutar un bloque de instrucciones mientras se dé una cierta instrucción. Su sintaxis de uso es:

```
while (<condición>)
{
    <instrucciones>
}
```

Su significado es el siguiente: Se evalúa la *condición* indicada, que ha de producir un valor lógico. Si es cierta (valor lógico **true**), se ejecutan las *instrucciones* y se repite el proceso de evaluación de la *condición* y ejecución de instrucciones, hasta que deje de serlo. Cuando sea falsa (**false**) se pasará a ejecutar la instrucción siguiente al **while**. En realidad puede ser una única instrucción o un bloque de instrucciones.

EJEMPLO:

```
int i = 0;
while (i < 5)
{
    Console.WriteLine(i);
    i++;
}
```

6.6.2.2. LA INSTRUCCIÓN DO...WHILE

La instrucción **do...while** es una variante del **while** que se usa así:

```
do
{
    <instrucciones>
}
while (<condición>)
```

La única diferencia del significado de **do...while** respecto al de **while** es que en vez de evaluar primero la *condición* y ejecutar las *instrucciones* sólo si es cierta, **do...while** primero ejecuta las *instrucciones* y luego mira la condición, para ver si se ha de repetir la ejecución de las mismas.

do ... while está especialmente destinado para los casos en los que haya que ejecutar las *instrucciones* al menos una vez, aun cuando la condición sea falsa desde el principio.

EJEMPLO:

```
int i = 0;
do
{
    Console.WriteLine(i);
    i++;
}
while (i < 5);
```

6.6.2.3. LA INSTRUCCIÓN FOR

La instrucción **for** es una variante de **while** que permite reducir el código necesario para escribir los tipos de bucles más comúnmente usados en programación. Su sintaxis es:

```
for (<inicialización>;<condición>;<modificación>)
{
    <instrucciones>
}
```

El significado de esta instrucción es el siguiente: se ejecutan las instrucciones de la *inicialización*, que suelen usarse para definir e inicializar variables que luego se usarán en las *instrucciones*. Luego se evalúa la *condición*, y si es falsa se continúa ejecutando por la instrucción siguiente al **for**; mientras que si es cierta se ejecutan las *instrucciones* indicadas, luego se ejecutan las instrucciones de *modificación* -que como su nombre indica suelen usarse para modificar los valores de variables que se usen en las *instrucciones*- y luego se reevalúa la *condición* repitiéndose el proceso hasta que ésta última deje de ser cierta.

EJEMPLO:

```
for (int i = 0; i <= 10; i = i + 2)
{
    Console.WriteLine(i);
}
```

6.6.2.4. LA INSTRUCCIÓN FOREACH

La instrucción **foreach** permite recorrer todos los elementos de una colección desde el primero al último. La sintaxis es la siguiente:

```
foreach (<tipoElemento> <elemento> in <colección>)
{
    <instrucciones>
}
```

EJEMPLO:

```
//Definimos e inicializamos un array de enteros
int[] nums = new int[] { 4, 2, 5, 7, 3, 7, 8 };
foreach (int i in nums)
//i es una variable local en el ámbito del foreach
{
    Console.WriteLine(i);
}
Console.ReadKey();
```

6.6.2.5. INSTRUCCIONES DE SALTO

En este apartado vamos a ver todas las instrucciones de salto que ofrece C#:

- ❖ **break**
- ❖ **continue**
- ❖ **goto**
- ❖ **return**

No obstante, sólo se recomienda el uso de **continue** y **return**. Y en todo caso, de **break** en los **switch**. En el resto de los casos es mejor prescindir de estas instrucciones.

BREAK

La instrucción **break** finaliza la ejecución del bucle contenedor más próximo o de la instrucción **switch** en la que aparezca. El control se pasa a la instrucción que hay a continuación de la instrucción finalizada, si existe.

EJEMPLO:

```
for (int i = 1; i <= 100; i++)
{
    if (i == 5)
    {
        break;
    }
    Console.WriteLine(i);
}
Console.WriteLine("Pulsa cualquier tecla para salir.");
Console.ReadKey();
```

En este ejemplo, la instrucción condicional contiene un contador que se supone que cuenta de 1 a 100. Pero la instrucción **break** finaliza el bucle después de 4 recuentos.

La instrucción **break** puede usarse para finalizar la ejecución en cualquier bucle (**while**, **do...while**, **for** y **foreach**).

CONTINUE

La instrucción **continue** pasa el control a la siguiente iteración de la instrucción de iteración envolvente en la que aparece.

EJEMPLO:

```
for (int i = 1; i <= 10; i++)
{
    if (i < 9)
    {
        continue;
    }
    Console.WriteLine(i);
}

Console.WriteLine("Pulsa cualquier tecla para salir.");
Console.ReadKey();
```

En este ejemplo se inicializa un contador para contar del 1 al 10. Si se usa la instrucción **continue** junto con la expresión ($i < 9$), se omiten las instrucciones comprendidas entre **continue** y el final del cuerpo de **for** en las iteraciones donde i es menor que 9. En las dos últimas iteraciones del bucle **for** (donde $i == 9$ e $i == 10$), la instrucción **continue** no se ejecuta y el valor de i se imprime en la consola.

GOTO

La instrucción **goto** transfiere el control del programa directamente a una instrucción con etiqueta.

Un uso común de **goto** consiste en transferir el control a una etiqueta de *switch case* específica o a la etiqueta predeterminada en una instrucción **switch**.

EJEMPLO 1:

```
Console.WriteLine("Tamaño del café: 1=Pequeño 2=Mediano 3=Grande");
Console.Write("Introduce tu elección: ");
string s = Console.ReadLine();
int n = Convert.ToInt32(s); // Conversión de tipos
int coste = 0;
switch (n)
{
    case 1:
        coste += 25;
        break;
    case 2:
        coste += 25;
        goto case 1;
    case 3:
        coste += 50;
        goto case 1;
    default:
        Console.WriteLine("Selección no válida.");
}
```

```
        break;
    }
    if (coste != 0)
        Console.WriteLine("Introduce " + coste + " céntimos.\n");

    Console.WriteLine("Pulsa cualquier tecla para salir.");
    Console.ReadKey();
```

La instrucción **goto** también es útil para salir de bucles más o menos complejos:

EJEMPLO 2:

```
//Definimos e inicializamos un array de enteros
int[] nums = new int[] { 4, 2, 5, 7, 3, 7, 8 };

Console.Write("Introduce el número que buscas: ");
string numBuscado = Console.ReadLine();

foreach (int i in nums)
//i es una variable local en el ámbito del foreach
{
    if (i == Convert.ToInt32(numBuscado))
        goto Encontrado;
    Console.WriteLine(i);
}

Console.WriteLine("\nEl número " + numBuscado + " NO ha sido
encontrado.");
goto Fin;

Encontrado:
    Console.WriteLine("\nEl número " + numBuscado + " SÍ ha sido
encontrado.");

Fin:
    Console.ReadKey();
```

RETURN

Se deja su estudio para más adelante, cuando se vean los procedimientos y funciones.

6.7. CASTING O CONVERSIÓN DE TIPOS

En C# después de declarar una variable, no se puede volver a declarar ni se le puede asignar un valor de otro tipo a menos que ese tipo sea convertible de forma implícita al tipo de la variable.

Por ejemplo, **string** no se puede convertir de forma implícita a **int**. Por tanto, después de declarar *i* como un valor *int*, no se le puede asignar la cadena "Hola".

No obstante, en ocasiones es posible que se necesite copiar un valor en una variable o parámetro de método de otro tipo. Por ejemplo, es posible que tengamos una variable de entero que se necesita pasar a un método cuyo parámetro es de tipo *double*. Estos tipos de operaciones se denominan **conversiones de tipos**. En C#, se pueden realizar las siguientes conversiones de tipos:

- 1) Conversiones implícitas
- 2) Conversiones explícitas
- 3) Conversiones definidas por el usuario
- 4) Conversiones con clases del asistente

A continuación vamos a centrarnos en las dos primeras. Las dos últimas las dejaremos para temas posteriores.

1) Conversiones implícitas

No se requiere ninguna sintaxis especial porque la conversión siempre es correcta y no se perderá ningún dato. Los ejemplos incluyen conversiones de tipos enteros más pequeños a más grandes.

EJEMPLO:

```
// Conversión implícita.  
// Una variable de tipo long puede albergar variables  
// de tipo entero sin perder precisión  
int num = 2147483647;  
long bigNum = num;
```

Para los tipos numéricos integrados, se puede realizar una conversión implícita cuando el valor que se va a almacenar se puede encajar en la variable sin truncarse ni redondearse. Para los tipos enteros, esto significa que el intervalo del tipo de origen es un subconjunto apropiado del intervalo para el tipo de destino. Por ejemplo, una variable de tipo **long** (entero de 64 bits) puede almacenar cualquier valor que un tipo **int** (entero de 32 bits) pueda almacenar.

2) Conversiones explícitas

Las conversiones explícitas requieren una **expresión Cast**. La conversión es necesaria si es posible que se pierda información en la conversión, o cuando es posible que la conversión no sea correcta por otros motivos. Entre los ejemplos típicos están la conversión numérica a un tipo que tiene menos precisión o un intervalo más pequeño.

Una **conversión de tipos** es una manera de informar explícitamente al compilador de que se pretende realizar la conversión y se es consciente de que se puede producir pérdida de datos o la conversión de tipos puede fallar en el tiempo de ejecución.

EJEMPLO:

El siguiente programa convierte un tipo **double** en un tipo **int**. El programa no se compilará sin la conversión.

FORMA 1:

```
double x = 1234.7;
int a;
// Casting de un double a un int. Sólo se puede utilizar con tipos
// numéricos como float, double o decimal
a = (int)x;
System.Console.WriteLine(a);
```

FORMA 2:

```
double x = 1234.7;
int a;
// Casting de un double a un int
a = Convert.ToInt32(x);
System.Console.WriteLine(a);
```

FORMA 3:

```
double x = 1234.7;
int a;
// Casting de un string a un int
a = int.Parse(x.ToString());
System.Console.WriteLine(a);
```

6.8. MATRICES (ARRAYS)

Un **matriz o array** es un conjunto de elementos contiguos, todos del mismo tipo, que comparten un nombre común, a los que se puede acceder por la posición (*índice*) que ocupa cada uno de ellos dentro de la matriz.

La representación de las matrices se hace mediante *índices* y pueden tener una o varias dimensiones. Así tendremos matrices unidimensionales, bidimensionales (también llamadas *tablas*), tridimensionales...

En C#:

- El primer índice siempre vale 0 y llega hasta un número menos que el tamaño de la matriz.
- El *número de dimensiones* y la *longitud* de cada dimensión se establecen al crear la instancia de matriz. No se pueden cambiar estos valores durante la vigencia de la instancia.
- Los valores predeterminados de los elementos numéricos de matriz se establecen en cero y los elementos de referencia se establecen en null.
- Los elementos de una matriz puede ser *cualquier tipo*, incluido un tipo de matriz.

6.8.1. Matrices Unidimensionales

La declaración de una matriz unidimensional se realiza especificando el nombre de la matriz, el número de elementos de la misma y el tipo de éstos.

Para declarar e inicializar los elementos de una matriz, se puede hacer de varias formas, tal y como se aprecia en el siguiente ejemplo:

```
//FORMA 1
//Declaro el array e indico el número de elementos que va a tener
int[] array1=new int[5];
//Le asigno sus valores elemento a elemento
array1[0] = 1;
array1[1] = 3;
array1[2] = 5;
array1[3] = 7;
array1[4] = 7;

//FORMA 2
//Declaro el array
//Le asigno al array todos los valores de sus elementos
int[] array2;
array2 = new int[] { 1, 3, 5, 7, 9 };

//FORMA 3
//Declaro y asigno valores al array en la misma sentencia
int[] array3 = new int[] { 1, 3, 5, 7, 9 };
```

```
//FORMA 4
//Declaro y asigno valores al array sin usar la expresión new
int[] array4 = { 1, 3, 5, 7, 9 };
```

6.8.2. Matrices Multidimensionales

Se trata de matrices que tienen dos o más dimensiones. Vamos a ver como trabajar con ellas.

1) Declaración e Inicialización

Para declarar e inicializar los elementos de una matriz multidimensional, al igual que con las unidimensionales, se puede hacer de varias formas:

```
//FORMA 1
//Declaro el array e indico el número de elementos que va a tener
//cada dimensión
int[,] array = new int[4, 2]; //Bidimensional, 4 filas y 2 columnas
//Le asigno sus valores elemento a elemento
array[0, 0] = 1;
array[0, 1] = 2;
array[1, 0] = 3;
array[1, 1] = 4;
array[2, 0] = 5;
array[2, 1] = 6;
array[3, 0] = 7;
array[3, 1] = 8;
```

```
//FORMA 2
//Declaro el array
//Le asigno al array todos los valores de sus elementos
int[,] array2D;
array2D = new int[3, 2] { { 1, 2 }, { 3, 4 }, { 5, 6 } };
```

```
//FORMA 3
//Declaro y asigno valores al array en la misma sentencia
int[,] array2D = new int[3, 2] { { 1, 2 }, { 3, 4 }, { 5, 6 } };
```

```
//FORMA 4
//Declaro y asigno valores al array sin especificar la longitud de
//cada dimensión

int[,] array2Da = new int[,] { { 1, 2 }, { 3, 4 }, { 5, 6 } };
```

```
//FORMA 5
//Declaro y asigno valores al array sin usar la expresión new
int[,] array2Da = { { 1, 2 }, { 3, 4 }, { 5, 6 } };
```

2) Recorrido por sus Elementos

Para mostrar o pedir el valor de cada uno de los elementos que conforman una matriz multidimensional, podemos utilizar bucles **for anidados**, o bien, un **foreach**:

EJEMPLO con FOR anidados para una matriz de 2 dimensiones:

```
//Recorremos todos los elementos del array con 2 for anidados
for (int i = 0; i < array2D.GetLength(0); i++)
//Recorremos la 1ª dimensión
{
    Console.WriteLine("FILA {0}\n", i + 1);
    for (int j = 0; j < array2D.GetLength(1); j++)
    //Recorremos la 2ª dimensión
    {
        Console.Write(array2D[i, j] + " ");
    }
    Console.WriteLine("");
}
```

EJEMPLO con FOREACH:

```
foreach (var i in array2D)
{
    Console.WriteLine("{0}\t", i);
}
```

3) Propiedades y Métodos de Interés

A la hora de trabajar con matrices, nos pueden ser de utilidad las siguientes propiedades y métodos:

- ❖ **Rank**: propiedad que devuelve el rango (número de dimensiones) de la matriz. Por ejemplo, una matriz unidimensional devuelve 1, una matriz bidimensional devuelve 2, y así sucesivamente.
- ❖ **Length**: propiedad que devuelve el número total de elementos (de todas las dimensiones) del array. En el caso de las *matrices escalonadas*, lo que nos devuelve es el número de matrices interiores que hay dentro de la matriz.
- ❖ **GetLength (int dimension)**: método que nos devuelve un entero de 32 bits que representa el número de elementos de la *dimensión* especificada del array.

EJEMPLO:

```
//Uso de Length, GetLength(dimensión), Rank
var allLength = array3D.Length; //Número de elementos del array
var total = 1;

Console.WriteLine
($"\\nEl array3D tiene array3D.Rank={0} dimensiones", array3D.Rank);

for (int i = 0; i < array3D.Rank; i++)
//Rank devuelve el número de dimensiones
{
    Console.WriteLine
    ("Dimensión {0} = {1} elementos", i, array3D.GetLength(i));
    total *= array3D.GetLength(i); //Longitud de esa dimensión
}
Console.WriteLine ("array3D.Length =
{0}\\ny\\narray3D.GetLength(0)*array3D.GetLength(1)*array3D.GetLength
(2))={1}", allLength, total);

Console.ReadLine();
```

6.8.3. Matrices Escalonadas

Una **matriz escalonada** es una matriz cuyos elementos son matrices, posiblemente de diferentes tamaños. Una matriz escalonada se denomina a veces "*matriz de matrices*". En los ejemplos siguientes, se muestra cómo declarar, inicializar y acceder a matrices escalonadas.

```
//FORMA 1
//Declaro el array e indico el número de elementos que va a tener.
//En nuestro caso vamos a tener 3 elementos, es decir, 3 matrices
//de elementos
int[][] matrizDeMatrices=new int[3][];
//Inicializo los elementos de cada matriz
matrizDeMatrices[0] = new int[5];
matrizDeMatrices[1] = new int[4];
matrizDeMatrices[2] = new int[2];
```

Como se ve en el ejemplo anterior, cada elemento de la matriz de matrices es una *matriz unidimensional* de enteros. La primera matriz es de 5 enteros, la segunda de 4 y la tercera de 2.

```
//FORMA 2
//Declaro el array e indico el número de elementos que va a tener
//En nuestro caso vamos a tener 3 elementos, es decir, 3 matrices
//de elementos
int[][] matrizDeMatrices=new int[3][];
//Inicializo los elementos de cada matriz con los valores que va a
//tener, por lo que no hace falta especificar el tamaño de cada
//matriz
matrizDeMatrices[0] = new int[] { 1, 3, 5, 7, 9 };
matrizDeMatrices[1] = new int[] { 0, 2, 4, 6 };
matrizDeMatrices[2] = new int[] { 11, 22 };
```

```
//FORMA 3
//Declaro el array y lo inicializo con los valores de los elementos
//de cada matriz en la misma instrucción
int[][] matrizDeMatrices=new int[][]
{
    new int[] { 1, 3, 5, 7, 9 },
    new int[] { 0, 2, 4, 6 },
    new int[] { 11, 22 }
};
```

```
//FORMA 4
//Forma abreviada
int[][] matrizDeMatrices=
{
    new int[] { 1, 3, 5, 7, 9 },
    new int[] { 0, 2, 4, 6 },
    new int[] { 11, 22 }
};
```

Una matriz escalonada es una matriz de matrices y, por consiguiente, sus elementos son tipos de referencia y se inicializan en *null*.

Podemos acceder a los elementos individuales de la matriz de la siguiente forma:

```
// Asignamos 77 al segundo elemento ([1]) del primer array ([0])
matrizDeMatrices [0][1] = 77;

// Asignamos 88 al segundo elemento ([1]) del tercer array ([2])
matrizDeMatrices [2][1] = 88;
```

Además, se pueden mezclar *matrices multidimensionales y escalonadas*. La siguiente es una declaración e inicialización de una matriz escalonada que contiene tres elementos de matriz bidimensional de tamaños diferentes:

EJEMPLO 1:

```
int[,] mDeMatricesBidimensionales = new int[3][,]
{
    new int[,] { {1,3}, {5,7} }, //matriz de 2x2
    new int[,] { {0,2}, {4,6}, {8,10} }, //matriz de 3x2
    new int[,] { {11,22}, {99,88}, {0,9} } //matriz de 3x2
};
System.Console.Write("{0}", mDeMatricesBidimensionales[0][1, 0]);
```

En el siguiente ejemplo declaramos y asignamos valores a todos los elementos de una *matriz escalonada de matrices unidimensionales*. A continuación recorreremos la matriz escalonada mostrando su contenido.

EJEMPLO 2:

```
// Declaro la matriz escalonada. Va a ser una matriz con 2 matrices
//en su interior
int[][] arr = new int[2][];

// Inicializo la primera matriz interior
arr[0] = new int[5] { 1, 3, 5, 7, 9 };
// Inicializo la segunda matriz interior
arr[1] = new int[4] { 2, 4, 6, 8 };

// Muestro los elementos
for (int i = 0; i < arr.Length; i++) //Nº de matrices interiores
{
    //Escribo Elementos(i), indicando i la matriz interior
    //que voy a visualizar
    System.Console.Write("Elementos({0}): ", i);

    for (int j = 0; j < arr[i].Length; j++)//Nº de elementos de
        //cada matriz interior
    {
        //Muestro todos los valores que hay dentro de cada
        //matriz interior. Para ello muestro el valor del
        //elemento[i][j] y a continuación un espacio
        System.Console.Write("{0} ", arr[i][j]);
    }
    System.Console.WriteLine();
}
System.Console.WriteLine("Pulsa cualquier tecla para salir.");
System.Console.ReadKey();
```

6.9. MÉTODOS

En POO, un **método** es un bloque de código que contiene una serie de instrucciones. Un programa hace que se ejecuten las instrucciones al llamar al método y especificando los argumentos de método necesarios. En C#, todas las instrucciones ejecutadas se realizan en el contexto de un método.

Ya hemos visto un método: el método **Main()**. Aunque C# permite poner todo el código en el método Main(), probablemente queramos diseñar nuestras clases para definir más de un método.

En los siguientes apartados vamos a aprender a crear métodos que devuelven datos y otros que no. A los primeros los llamaremos **funciones** y a los segundos, **procedimientos**.

Igualmente, aprenderemos a pasar *parámetros* a los métodos y a saber estructurar un método para hacer que nuestras aplicaciones sean *modulares*.

6.9.1. Estructura de un Método

Cualquier método está compuesto de las siguientes partes:

- ❖ **Tipo devuelto:** tipo de dato del valor que el método devolverá al terminar su ejecución. Si deseamos que el método no retorne nada, en ese caso utilizaremos la palabra *void*.
- ❖ **Nombre del método** (único en el ámbito): nombre con el que se identifica y se distingue de otros. No podrá haber otro método con ese nombre (salvo sobrecarga o polimorfismo en POO).
- ❖ **Lista de parámetros:** especificación del conjunto de argumentos (pueden ser cero, uno o más) que el método debe recibir para realizar su tarea. Se encierran entre paréntesis y se separan por comas. Los paréntesis vacíos indican que el método no requiere parámetros.
- ❖ **Cuerpo del método:** es el bloque de instrucciones que compone el código del método.

Además, los métodos tienen otras partes opcionales, como las listas de atributos y los modificadores de ámbito, que se explicarán en el siguiente tema, cuando nos adentremos en la programación orientada a objetos.

EJEMPLO:

```
class Program
{
    static void Main(string[] args)
    {
        Console.Write("¿Cómo te llamas?: ");
        string miNombre = Console.ReadLine();
        Console.Write("¿Qué edad tienes?: ");
        int miEdad = int.Parse(Console.ReadLine());

        //Llamada a los procedimientos
        Imprimir(miNombre);
        ImprimirSiMayoriaEdad(miEdad);

        Console.ReadKey();
    }

    //PROCEDIMIENTO = método que no devuelve nada
    //Recibe un parámetro de tipo string por valor
    public static void Imprimir(string nombre)
    {
        Console.WriteLine("Hola " + nombre);
    }

    //PROCEDIMIENTO = método que no devuelve nada.
    //Recibe un parámetro de tipo int por valor
    public static void ImprimirSiMayoriaEdad(int edad)
    {
        string cadena = "eres mayor de edad";
        string cadAuxiliar;

        //Llamamos a la función VerificarEdad
        if (VerificarEdad(edad))
            cadAuxiliar = "Sí ";
        else
            cadAuxiliar = "No ";

        Console.WriteLine(cadAuxiliar + cadena);
    }

    //FUNCIÓN = método que devuelve uno o más valores
    //Recibe un parámetro de tipo int por valor
    public static bool VerificarEdad(int edad)
    {
        if (edad >= 18)
            return true;
        else
            return false;
    }
}
```

6.9.2. Tipos de Métodos en C#

A continuación vamos a ver cuáles son los tipos de métodos que podemos distinguir en C#:

- ❖ **Procedimientos:** métodos de tipo *void*, es decir, que no devuelven ningún valor.
- ❖ **Funciones:** devuelven uno o más valores.
- ❖ **Procedimientos de propiedades:** permiten manejar las propiedades de los objetos creados.
- ❖ **Procedimientos de operador:** se utilizan para modificar el funcionamiento de un operador cuando se aplica a una clase o estructura.

De momento no vamos a profundizar en los dos últimos, dejando su estudio para temas posteriores, cuando veamos la POO.

6.9.3. Métodos de Expresión Corporal

Existen ocasiones, en las que deseamos ejecutar alguna operación sencilla, o bien, una única operación en nuestro código, por lo que no deseamos declarar un método completo, ya que esto podría suponer la creación de mucho código.

Afortunadamente, en C#, contamos con una forma simplificada de escribir dichos métodos, que al igual que un método normal, nos permite retornar un tipo de dato, y a la vez, pasar parámetros.

La definición de estos métodos, es de la siguiente forma:

tipoDatoRetorno NombreMétodo (<parámetros>) => CuerpoMétodo

EJEMPLO:

```
//PROCEDIMIENTO de Expresión Corporal
public static void Imprimir(string nombre) =>
    Console.WriteLine("Hola " + nombre);

//FUNCIÓN de Expresión Corporal
public static bool VerificarEdad(int edad)=> edad>=18? true:false;
```

Hemos modificado dos de los procedimientos del ejemplo anterior reduciendo visiblemente el código. Obsérvese que lo único que necesitamos es poner el símbolo => y la palabra **return** desaparece.

6.9.3. Métodos que Devuelven Varios Valores

Hay ocasiones en las que nos puede interesar retornar múltiples valores de un método. Para ello tendremos que optar por una de estas dos opciones: pasar los **valores por referencia** o utilizar **tuplas**.

6.9.3.1. PASO DE PARÁMETROS POR REFERENCIA

De forma predeterminada, cuando un objeto se pasa a un método, se pasa una copia en lugar del propio objeto. Por lo tanto, los cambios realizados en el argumento no tienen ningún efecto en la copia original del método de llamada. Esto es lo que se conoce como **paso por valor**.

Si en su lugar lo que se pasa es una referencia al objeto, entonces estaremos haciendo un **paso por referencia**. Es decir, el método no recibirá el objeto concreto, recibirá un argumento que indicará la ubicación del objeto. Por lo tanto, si se modifica el objeto mediante esta referencia, el cambio se reflejará en el argumento del método de llamada.

Para poder pasar un objeto por **referencia** se ha utilizar la palabra clave **ref** o **out**, tanto en la llamada como en la declaración del método.

Las variables que se han pasado como argumentos **out** no tienen que inicializarse antes de pasarse en una llamada al método. En cambio, si se utiliza la palabra **ref**, si tienen que inicializarse antes.

EJEMPLO con **ref**:

```
class Program
{
    static void Main(string[] args)
    {
        int valor1=0, valor2=0;

        //Llamada al método con el paso de los parámetros por
        //referencia
        RegresarValores(ref valor1, ref valor2);

        Console.WriteLine(valor1 + " " + valor2);
        Console.ReadKey();
    }

    //Método que recibe 2 parámetros por referencia
    public static void RegresarValores(ref int num1, ref int num2)
    {
        num1 = 45;
        num2 = 28;
    }
}
```

EJEMPLO con **out**:

```
class Program
{
    static void Main(string[] args)
    {
        int valor1, valor2;

        //Llamada al método con el paso de los parámetros por
        //referencia
        RegresarValores(out valor1, out valor2);

        Console.WriteLine(valor1 + " " + valor2);
        Console.ReadKey();
    }

    //Método que recibe 2 parámetros por referencia
    public static void RegresarValores(out int num1, out int num2)
    {
        num1 = 45;
        num2 = 28;
    }
}
```

6.9.3.2. TUPLAS

Una **tupla**, en resumidas cuentas, es una colección pequeña de valores. Para definir una tupla únicamente tendremos que definir el tipo de dato de cada uno de los valores que es retornado.

Es importante recalcar, que esta característica, está disponible a partir de la versión de C# 7.0.

Una vez hecho esto, podremos regresar más de un valor como parte de un método, tal como en el siguiente ejemplo:

```
class Program
{
    static void Main(string[] args)
    {
        int valor1, valor2;
        (valor1, valor2) = RegresarValores();
        Console.WriteLine(valor1 + " " + valor2);
    }

    //FUNCIÓN que devuelve una tupla con 2 valores enteros
    public static (int, int) RegresarValores()
    {

```

```
        return (45, 28);  
    }  
}
```

6.9.4. Métodos Anidados o Funciones Locales

A partir de C# 7.0, podemos utilizar los llamados métodos anidados o funciones locales, que son métodos privados que están anidados dentro de otros métodos en nuestro código.

Esto nos permite encapsular funcionalidad que no queremos exponer a través de nuestro código, ya que este tipo de métodos es únicamente llamable desde el método que lo implementa.

EJEMPLO:

```
class Program  
{  
    static void Main(string[] args)  
    {  
        int numero = 4;  
        Console.WriteLine(EsPar(numero));  
        Console.ReadLine();  
    }  
  
    static string EsPar(int num)  
    {  
        //Llamo a la función local  
        string cadenaEsPar = DevuelveCadenaEsPar(num);  
  
        return cadenaEsPar;  
  
        //Función local  
        string DevuelveCadenaEsPar(int dato)  
        {  
            if (dato % 2 == 0)  
                return "Es PAR";  
            else  
                return "Es IMPAR";  
        }  
    }  
}
```

Como se puede apreciar, dentro del método llamado *EsPar*, tenemos otro método llamado *DevuelveCadenaEsPar*, el cual es el que nos permitirá llevar a cabo la operación para saber si un número es par o no.

Por otro lado, desde nuestro código en el método *Main*, podremos invocar al método *EsPar*, pero no al método *DevuelveCadenaEsPar*, ya que está fuera de su ámbito.

6.9.5. Más opciones para el Paso de Parámetros

Hay casos en los que nos podría venir bien definir un método con uno o más **parámetros opcionales**. También podría resultarnos interesante el no tener que pasar los parámetros en el mismo **orden** en el que aparecen en la declaración del método.

En los próximos apartados vamos a profundizar en estos dos casos.

6.9.5.1. PARÁMETROS OPCIONALES

Vamos a ver cómo se haría a través de un ejemplo.

Supongamos que un restaurante desea implementar un pequeño sistema, que permita a los consumidores calcular el precio total por persona de una comida con la propina incluida (si la hubiera). Codificaríamos un método similar al siguiente:

EJEMPLO:

```
static void Main(string[] args)
{
    //Llamamos a la función pasándole todos los parámetros
    Console.WriteLine("CON PROPINA-----");
    Console.WriteLine(CalcularImportePorPersona(37.65,2,1.50)+" euros.");
    Console.ReadLine();

    //Llamamos a la función sin pasarle la propina
    Console.WriteLine("SIN PROPINA-----");
    Console.WriteLine(CalcularImportePorPersona(37.65, 2) + " euros.");
    Console.ReadLine();
}

//Indicamos que el parámetro propina va a ser opcional al
//asignarle un valor en la declaración de la función
```

```
static double CalcularImportePorPersona(double importeComida, int
numPersonas, double propina = 0)
{
    double total = propina + importeComida;
    double totalPorPersona = total / numPersonas;
    return totalPorPersona;
}
```

Tal y como puede apreciarse en el ejemplo, lo único que hay que hacer para que un parámetro sea *opcional* es asignarle un valor en la declaración del método.

6.9.5.2. ARGUMENTOS CON NOMBRE

Podemos indicar el nombre de cada argumento que se le pasa al método junto con su valor, sin tener que seguir el mismo orden establecido en la declaración del método.

Veamos de nuevo el ejemplo del restaurante. Solamente cambiaría la forma en la que invocamos al método.

EJEMPLO:

```
//Llamamos a la función pasándole todos los parámetros desordenados
Console.WriteLine("CON PROPINA-----");

Console.WriteLine(CalcularImportePorPersona(importeComida:37.65,propina:1
.50,numPersonas:2) + " euros.");
Console.ReadLine();

//Llamamos a la función sin pasarle la propina y con los parámetros
//desordenados
Console.WriteLine("SIN PROPINA-----");

Console.WriteLine(CalcularImportePorPersona(numPersonas:2,importeComida:3
7.65) + " euros.");
Console.ReadLine();
```

6.10. MANIPULACIÓN DE CADENAS DE CARACTERES

Las cadenas de caracteres son muy importantes en el desarrollo de software. Todos los lenguajes de programación tienen herramientas para la manipulación primitiva de caracteres, pero C# cuenta con una colección de métodos especialmente útil.

En este apartado analizaremos todas las herramientas para trabajar con cadenas que hemos utilizado hasta el momento, y ampliaremos nuestro conocimiento al estudiar el conjunto de métodos disponibles para el procesamiento de cadenas.

6.10.1. Elementos de Formato

Un elemento de formato tiene esta sintaxis:

```
{índice[,alineación][:formato]}
```

Donde:

- ❖ El **índice** comienza en 0 y representa al argumento cuya representación se va a incluir en esta posición de la cadena.
- ❖ La **alineación** (*opcional*) es un entero con signo que indica la longitud total del campo en el que se inserta el argumento. Si está alineado a la derecha será un entero positivo, si no, estará alineado a la izquierda y será un entero negativo.
- ❖ El **formato** (*opcional*). Cadena que especifica el formato de la cadena de resultado del argumento correspondiente.

Vamos a ver su aplicación con varios ejemplos.

EJEMPLO 1:

```
//FORMAS de inicializar un objeto de la estructura DateTime
DateTime fecha = new DateTime();
fecha = DateTime.Now;

DateTime fecha2 = new DateTime(2021, 10, 2, 20, 36, 46);

string strFecha3 = "6/1/2016 6:34:53";
DateTime fecha3 = DateTime.Parse(strFecha3);

//02/10/2021 20:36:46
Console.WriteLine("ToLocalTime: " + fecha.ToLocalTime());

//sábado, 02 de octubre de 2021
Console.WriteLine("ToLongDateString: " + fecha.ToLongDateString());

//20:36:46
Console.WriteLine("ToLongTimeString: " + fecha.ToLongTimeString());

//02/10/2021
Console.WriteLine("ToShortDateString: " + fecha.ToShortDateString());

//20:36
Console.WriteLine("ToShortTimeString: " + fecha.ToShortTimeString());

//02/10/2021 20:36:46
Console.WriteLine("ToString: " + fecha.ToString());

//02/10/2021 18:36:46
```

```

Console.WriteLine("ToUniversalTime: " + fecha.ToUniversalTime());
Console.ReadLine();

//02/10/2021 08:36:46
string fechaString = String.Format("La fecha de hoy es: {0:dd/MM/yyyy
hh:mm:ss}", DateTime.Now);
Console.WriteLine(fechaString);

//sábado, 2 de octubre de 2021 20:36
fechaString = String.Format("La fecha de hoy es: {0:dddd, d\" de \"MMMM\"
de \"yyyy HH:mm}", DateTime.Now);
Console.WriteLine(fechaString);

```

La estructura **DateTime** nos permite trabajar con fechas. Esta clase tiene muchos métodos que ya nos permiten obtener la fecha como un string. Además, mediante el método **String.Format** podemos pasarle los elementos de formato que nos interesen. Para conocer cuáles son todos los elementos de formato aplicables a fechas, véase el documento *Formatos para fechas* incluido en la plataforma.

EJEMPLO 2:

```

object[] values = { 1603, 1794.68235, 2.45555 };
string result;
foreach (var value in values) {
    result = String.Format("{0,12:C2}    {0,12:E3}    {0,12:F4}    {0,12:N3}
{1,12:P2}\n", Convert.ToDouble(value), Convert.ToDouble(value) / 10000);
    Console.WriteLine(result);
}

// El ejemplo muestra una salida como ésta:
//      $1,603.00      1.603E+003      1603.0000      1,603.000      16.03 %
//
//      $1,794.68      1.795E+003      1794.6824      1,794.682      17.95 %
//
//      $2,46          2,456E+000          2,4556          2,456          0,02 %
//
//      moneda      exponencial      punto fijo      nº decimales      porcentaje

```

Como se aprecia en el ejemplo, también podemos usar elementos de formato para especificar el número de decimales y las posiciones que ocupará el argumento en cuestión.

Se puede consultar más información sobre los elementos de formato en <https://docs.microsoft.com/es-es/dotnet/api/system.string.format?view=net-5.0>.

6.10.2. Concatenación de cadenas

Existen tres formas de concatenar cadenas:

1. **Operador +**. Ya se ha visto en apartados y ejemplos anteriores.
2. El método **Concat** de la clase **String**. Se deja su estudio para el apartado 6.10.2.3. No obstante, en el siguiente ejemplo veremos su uso.
3. **Interpolación de cadenas**. Toda *expresión de cadena interpolada* comienza por el carácter **\$** seguido de una cadena de caracteres que irá entre comillas dobles. A su vez, en esa cadena de caracteres debe haber una o más expresiones de interpolación. Una *expresión de interpolación* se indica mediante una llave de apertura y de cierre (**{ y }**).

A continuación vamos a ver un ejemplo donde se muestra el uso indistinto de cada una de estas tres formas.

EJEMPLO:

```
static void Main(string[] args)
{
    string superheroe = "Superman";
    string procedencia = "Krypton";
    string superpoder = "super fuerza";

    //OPERADOR +
    string todosDatos = superheroe + " procede de " + procedencia + " y
tiene " + superpoder;
    Console.WriteLine(todosDatos);
    Console.ReadLine();

    //CONCAT
    string todosDatosConcat = String.Concat(superheroe, " procede de ",
procedencia, " y tiene ", superpoder);
    Console.WriteLine(todosDatosConcat);
    Console.ReadLine();

    //INTERPOLACIÓN
    string todosDatosInterpolation = $"{superheroe} procede de
{procedencia} y tiene {superpoder}";
    string ejemploInterpolacion = $"{ 3 + 5 } es igual a 8?";
    Console.WriteLine(todosDatosInterpolation);
    Console.WriteLine(ejemploInterpolacion);
    Console.ReadLine();
}
```

6.10.3. Clase **StringBuilder**

La clase **String** representa una cadena de caracteres *immutable*. Es decir, cada operación que se muestra para modificar un string realmente crea una nueva

cadena. Por ejemplo, el método **String.Concat** crea una nueva cadena con el resultado de la concatenación y ésta se le asigna a la cadena original.

En el caso de las rutinas que realizan una manipulación de cadenas extensa (como las aplicaciones que modifican una cadena varias veces en un bucle), la modificación de una cadena puede tener una penalización de rendimiento significativa. La alternativa es usar la clase **StringBuilder**, que es una clase de cadena *mutable*. Mutabilidad significa que una vez creada una instancia de la clase, se puede modificar anexando, quitando, reemplazando o insertando caracteres.

Un objeto `StringBuilder` mantiene un búfer para alojar las expansiones en la cadena. Los datos nuevos se anexan al búfer si hay espacio disponible; de lo contrario, se asigna un búfer nuevo y mayor, los datos del búfer original se copian en el nuevo búfer y, a continuación, los nuevos datos se anexan al nuevo búfer.

Pero no siempre se aconseja el uso de la clase `StringBuilder`, ya que la mejora en cuanto a rendimiento no es siempre significativa. Su uso se aconseja cuando se espera que la aplicación realice:

- ❖ Un *número desconocido de cambios* en una cadena en tiempo de diseño (por ejemplo, cuando se usa un bucle para concatenar un número aleatorio de cadenas que contienen datos proporcionados por el usuario).
- ❖ Un *número significativo de cambios* en una cadena.

En el resto de los casos, no traerá cuenta usar esta clase, ya que además carece de métodos de búsqueda como **IndexOf** o **StartsWith**.

A continuación se muestra un ejemplo sobre el funcionamiento y uso de la clase `StringBuilder`. En el siguiente enlace se puede profundizar más sobre esta clase: <https://docs.microsoft.com/es-es/dotnet/api/system.text.stringbuilder?view=net-5.0>.

EJEMPLO:

```
static void Main(string[] args)
{
    StringBuilder sb = new StringBuilder();
    StringBuilder sb2 = new StringBuilder("Hola soy Batman");

    //Añado texto sin saltos de línea
    sb.Append("Hola soy Batman");
    sb.Append(" y vivo en Gotham");
    Console.WriteLine(sb);
    Console.ReadLine();
}
```

```
//Añado texto y salto de línea al final
sb2.AppendLine(" y vivo en Gotham");
//Añado texto aplicando un elemento de formato
sb2.AppendFormat("Actualmente {0:dd/MM/yyyy}", DateTime.Now);
Console.WriteLine(sb2);
Console.ReadLine();

//Cadena interpolada donde muestro el nº de caracteres de sb2
Console.WriteLine($"La longitud de la cadena sb2 es {sb2.Length}");
Console.ReadLine();
}
```

6.10.2. Propiedades y Métodos de la clase String

En esta sección examinaremos los métodos más útiles de la clase **String**. Según la función que realice cada método, podremos incluirlo en una de las siguientes categorías de métodos:

- ❖ *Corrección de cadenas*
- ❖ *Análisis de cadenas*

6.10.2.1. CORRECCIÓN DE CADENAS

En esta sección veremos métodos para modificar cadenas de caracteres. Estos métodos en realidad crean una nueva cadena, en vez de modificar la original.

- ❖ **string ToUpper() y string ToLower()**: Devuelven, respectivamente, la cadena que resulte de convertir a mayúsculas o minúsculas la cadena sobre la que se aplican.
- ❖ **string TrimStart(), string TrimEnd() y string Trim()**: Eliminan los espacios del principio, del final y del principio y final de una cadena, respectivamente.
- ❖ **string Insert(int posición, string subcadena)**: Devuelve la cadena resultante de insertar la *subcadena* indicada en la posición especificada de la cadena sobre la que se aplica.
- ❖ **string Remove(int posición, int número)**: Devuelve la cadena resultante de eliminar el número de caracteres indicado que hubiese en la cadena sobre la que se aplica a partir de la posición especificada.
- ❖ **string Replace(string cadenaOld, string cadenaNew)**: Devuelve la cadena resultante de sustituir en la cadena sobre la que se aplica toda aparición de *cadenaOld* por *cadenaNew*.

- ❖ **String Concat (string cad1, string cad2):** Devuelve la cadena resultante de concatenar la cadena *cad1* y la cadena *cad2*.
- ❖ **string PadLeft (int longTotal, <char caracterRelleno>):** Devuelve una nueva cadena que alinea a la derecha los caracteres de la instancia desde la cadena desde la que se llama al método e inserta a la izquierda el *carácter de relleno* especificado hasta alcanzar la *longitud total* especificada. Si no se introduce ningún carácter de relleno simplemente deja espacios en blanco.
- ❖ **string PadRight (int longTotal, <char caracterRelleno>):** Devuelve una nueva cadena que alinea a la izquierda los caracteres de la instancia desde la cadena desde la que se llama al método e inserta a la derecha el *carácter de relleno* especificado hasta alcanzar la *longitud total* especificada. Si no se introduce ningún carácter de relleno simplemente deja espacios en blanco.

EJEMPLO:

```
string[] alumnos = { "Lorena", "Sandra", "José Andrés", "Federico", "Miguel Ángel",  
"Carolina"};  
  
int i = 0;  
foreach (var elemento in alumnos)  
{  
    // PadRight añade espacios al final para que tengan todos la misma longitud  
    Console.WriteLine(String.Format("alumnos[{0}]= {1}", i, elemento).PadRight(40));  
    i++;  
}  
Console.ReadLine();
```

6.10.2.2. ANÁLISIS DE CADENAS

Los métodos y propiedades que comentaremos a continuación nos permiten examinar una cadena de caracteres para, por ejemplo, extraer una sección (o subcadena) de la misma.

- ❖ **int Length:** Esta propiedad nos proporciona el número de caracteres de la cadena en cuestión.
- ❖ **string Substring(int posición, int número):** Devuelve la subcadena que comienza en la posición indicada y tiene el número de caracteres especificados. Si no se indica dicho número se devuelve la subcadena que va desde la posición indicada hasta el final de la cadena.

- ❖ **int IndexOf(string subcadena)**: Indica cuál es el índice de la primera aparición de la *subcadena* indicada dentro de la cadena en cuestión. La búsqueda de dicha *subcadena* se realiza desde el principio de la cadena, pero es posible indicar en un segundo parámetro opcional de tipo `int` cuál es el índice de la misma a partir del que se desea empezar a buscar. Del mismo modo, la búsqueda acaba al llegar al final de la cadena sobre la que se busca, pero pasando un tercer parámetro opcional de tipo `int` es posible indicar algún índice anterior donde terminarla. Nótese que es un método muy útil para saber si una cadena contiene o no alguna *subcadena* determinada, pues sólo si no la encuentra devuelve un `-1`.
- ❖ **int LastIndexOf(string subcadena)**: Funciona de forma similar a *IndexOf()* sólo que devuelve la posición de la última aparición de la subcadena buscada en lugar de devolver la de la primera.
- ❖ **string[] Split (char[] separadores)**: Este método nos ayuda a dividir una cadena en secciones delimitadas por los separadores.
- ❖ **bool StartsWith (string cadBuscada) y bool EndsWith (string cadBuscada)**: Determinan si una cadena empieza o termina con una subcadena específica, respectivamente.
- ❖ **int CompareTo (string cadComparada)**: Devuelve 0 si la cadena original es igual a *cadComparada*. Si ordenadas alfabéticamente es anterior a *cadComparada*, devuelve -1. En caso contrario, 1.
- ❖ **bool Contains (string cadBuscada)**: Devuelve *true* si la cadena desde la que se llama al método *Contains* contiene a la cadena *cadBuscada*.

EJEMPLO:

```
static void Main(string[] args)
{
    string villano = "Gru";
    string esbirros = "Minions";
    string frase = "Los minions ayudan a Gru a conquistar el mundo";

    //Concatenar cadenas
    Console.WriteLine("CONCAT:\nvillano={0} + tiene +
    esbirros={1}\n",villano, esbirros);
    string concatenar = String.Concat(villano, " tiene ", esbirros);
    Console.WriteLine(concatenar);
    Console.ReadLine();

    //Encontrar cadena
    Console.WriteLine("CONTAINS:\nCompruebo si la variable villano contiene
    la cadena \"ru\"");
    if (villano.Contains("ru"))
```

```
{
    Console.Write("\nSí la contiene porque villano={0}\n",
        villano);
}
Console.ReadLine();

//Encontrar cadena
Console.Write("Compruebo si la variable esbirros contiene la cadena
\"Minions\"");
if (esbirros.Contains("Minions"))
{
    Console.Write("\nSí la contiene porque esbirros={0}\n",
        esbirros);
}
Console.ReadLine();

//Obtener subcadena
Console.Write("SUBSTRING:\nExtraigo 7 caracteres a partir de la
posición 4 en:\n");
Console.Write("frase=\"Los minions ayudan a Gru a conquistar el
mundo\"");
string subcadena = frase.Substring(4, 7);
Console.Write("\n=>{0}\n", subcadena);
Console.ReadLine();

//Buscar si la cadena acaba en...
Console.Write("ENDSWITH:\nCompruebo si la variable frase acaba con
\"mundo\"");
if (frase.EndsWith("mundo"))
{
    Console.Write("\nSí acaba porque frase={0}\n", frase);
}
Console.ReadLine();

//Insertar una cadena en otra
Console.Write("INSERT:\nInserto en la cadena frase={0}", frase);
Console.Write("\nla cadena \" adorables\" en la posición 3 teniendo
en cuenta que la 1ª posición es la 0\n");
frase = frase.Insert(3, " adorables");
Console.WriteLine(frase);
Console.ReadLine();

//Encontrar la última posición en la que aparece una cadena
Console.Write("LASTINDEXOF:\nBuscamos la última posición en la que
aparece la cadena \" a \"");
int posicion = frase.LastIndexOf(" a ");
Console.Write("\nen la frase={0}\nLa posición es la
{1}", frase, posicion);
Console.ReadLine();

//Encontrar la primera posición de una cadena
Console.Write("\nINDEXOF:\nBuscamos la primera posición en la que
aparece la cadena \"mundo\"");
posicion = frase.IndexOf("mundo");
Console.Write("\nen la frase={0}\nLa posición es la {1}", frase,
    posicion);
Console.ReadLine();
```

```
//Borrar una subcadena
Console.WriteLine("\nREMOVE:\nBorramos la palabra \"adorables\" de la frase.");
Console.WriteLine("\nPara ello, obtenemos la posición en la que se encuentra esta palabra y borramos 10 caracteres\n");
int posicion2 = frase.IndexOf("adorables");
frase = frase.Remove(posicion2, 10);
Console.WriteLine(frase);
Console.ReadLine();

//Reemplazar cadena
Console.WriteLine("REPLACE:\nReemplazamos la cadena \"el mundo\" por la cadena \"la luna\"\n");
frase = frase.Replace("el mundo", "la luna");
Console.WriteLine(frase);
Console.ReadLine();

//Dividir cadena
Console.WriteLine("SPLIT:\nSeparamos la frase en palabras, usando el delimitador ' '\n");
var palabras = frase.Split(' ');
foreach (var palabra in palabras)
{
    Console.WriteLine(palabra);
}
Console.ReadLine();

//Pasar mayúsculas y minúsculas
Console.WriteLine("TOUPPER Y TOLOWER:\n");
string GRU = villano.ToUpper();
string esbirros2 = esbirros.ToLower();
Console.WriteLine("{0} y {1}",villano, GRU);
Console.WriteLine("{0} y {1}", esbirros,esbirros2);
Console.ReadLine();

//Podar cadenas
Console.WriteLine("TRIMSTART, TRIMEND, TRIM:\n");
Console.WriteLine("Eliminar espacios del principio, del final y de ambos lados");
string espaciosDelanteros = " palabra";
string espaciosTraseros = "palabras ";
string espacios = " palabras ";

Console.WriteLine("\n{0}.\n{1}.",espaciosDelanteros, espaciosDelanteros.TrimStart());
Console.WriteLine("\n{0}.\n{1}.",espaciosTraseros, espaciosTraseros.TrimEnd());
Console.WriteLine("\n{0}.\n{1}.",espacios,espacios.Trim());
Console.ReadLine();

//Determinar la longitud de una cadena
Console.WriteLine("LENGTH:\n");
Console.WriteLine("La longitud de la frase:\"{0}\" es {1}", frase, frase.Length);
Console.ReadLine();

//Determinar qué cadena está antes alfabéticamente
Console.WriteLine("\nCOMPARETO:");
```

```
string ciudad1 = "ALICANTE";
string ciudad2 = "Alicante";

if (ciudad1.CompareTo(ciudad2) == -1)
    Console.WriteLine("\n{0} está antes que {1}",ciudad1,ciudad2);
else if (ciudad1.CompareTo(ciudad2)==1)
    Console.WriteLine("\n{0} está antes que {1}", ciudad2, ciudad1);
else
    Console.WriteLine("\n{0} es la misma cadena que {1}", ciudad1,
        ciudad2);
Console.ReadLine();
}
```

6.10.3. Expresiones Regulares

Casi toda la programación con cadenas de caracteres puede llevarse a cabo con las herramientas que hemos visto hasta ahora. Sin embargo, a veces surge la necesidad de realizar complicadas comprobaciones en los caracteres de una cadena, tal vez para validar alguna entrada del usuario. Ciertamente podemos usar los métodos para trabajar con cadenas que vimos antes, pero también tenemos la alternativa de emplear **expresiones regulares**.

He aquí algunos ejemplos de pruebas complicadas:

- El código de un producto debe consistir en una letra mayúscula seguida de un dígito.
- El código del personal debe consistir en cualquier cantidad de dígitos.
- La calificación de un estudiante debe consistir en A, B o C por su logro, seguida de 1, 2 o 3 por su esfuerzo.

Las expresiones regulares nos permiten construir un patrón con el cual comparar nuestros datos. Las cadenas de expresión regular contienen símbolos especiales que describen las posibilidades de búsqueda para cada carácter. He aquí una lista de los más comunes:

Símbolo	Acción
.	Busca coincidencias con cualquier carácter.
^	Busca coincidencias con el inicio de una cadena.
\$	Busca coincidencias con el final de una cadena.
[]	Encierra las alternativas para búsquedas para un solo carácter. Por ejemplo, [ABC] indica que un carácter puede ser A, B o C.
{}	Encierra una cuenta repetitiva. Por ejemplo, {5} busca una coincidencia exacta con cinco ocurrencias del elemento anterior.

+	Busca coincidencias con una o más ocurrencias del elemento anterior.
-	Expresa un rango de caracteres. Por ejemplo, A-Z busca coincidencias con cualquier letra mayúscula.
\	Provee al siguiente carácter su significado normal. Por ejemplo, \. buscará coincidencias con el carácter . en vez de buscarlas con cualquier carácter.

6.10.3.1. ISMATCH

Las expresiones regulares tienen varios usos; en esta sección nos concentraremos en comprobar que el contenido de una cadena cumpla con nuestros requerimientos.

El método **IsMatch** (un método estático de la clase `Regex`) nos permite comprobar las coincidencias entre una cadena de datos y una cadena de expresión regular. El valor que devuelve es *true* o *false*, dependiendo de si se encontró una coincidencia o no. Para utilizar este método debemos especificar un nuevo espacio de nombres en la parte superior de nuestro programa: `System.Text.RegularExpressions`.

EJEMPLO:

//VARIOS EJEMPLOS DE EXPRESIONES REGULARES

```
//1) La calificación de un estudiante debe consistir en:  
//A, B o C por su logro, seguida de 1, 2 ó 3 por su esfuerzo.  
string[] calificaciones = { " a4", "A2", "a3", "c6", "D 9" };  
foreach (var elemento in calificaciones)  
{  
    if (Regex.IsMatch(elemento, "^[ABC][123]$"))  
        Console.WriteLine(String.Format($"La calificación \"{elemento}\" es  
correcta").PadRight(40));  
    else  
        Console.WriteLine(String.Format($"La calificación \"{elemento}\" NO es  
correcta").PadRight(40));  
}  
  
Console.WriteLine("\n\n");  
  
//2) El código de un producto debe empezar por una mayúscula seguida de un dígito  
string[] codProductos = { " a4", "A4", "a4", "c6", "D 9" };  
foreach (var elemento in codProductos)  
{  
    if (Regex.IsMatch(elemento, "^[A-Z][0123456789]$"))  
        Console.WriteLine(String.Format($"El código \"{elemento}\" es  
correcto").PadRight(40));  
    else  
        Console.WriteLine(String.Format($"El código \"{elemento}\" NO es  
correcto").PadRight(40));  
}
```

```
Console.WriteLine("\n\n");

//3) Un nº de teléfono de Córdoba capital debe tener 9 dígitos y empezar por 957
//Suponemos que puede haber un espacio entre el prefijo y los 6 dígitos restantes
string[] telfs = { "957404040", "957 123434", "956777777", "9571234" };
foreach (var elemento in telfs)
{
    if (Regex.IsMatch(elemento, "^957[0123456789]{6}$") ||
        Regex.IsMatch(elemento, "^957 [0123456789]{6}$"))
        Console.WriteLine(String.Format($"El teléfono \"{elemento}\" es correcto").PadRight(40));
    else
        Console.WriteLine(String.Format($"El teléfono \"{elemento}\" NO es correcto").PadRight(40));
}
Console.ReadLine();
```

He aquí algunas expresiones regulares y las cadenas con las que buscan coincidencias:

Expresión	Coincidencias buscadas
"^..\$"	Busca coincidencias con dos caracteres cualesquiera, como AD, 45, //
"^A\$"	Busca coincidencias con cualquier carácter seguido de una A
"^A-Z\$"	Busca coincidencias con cualquier letra mayúscula
"^A-Za-z+"	Busca coincidencias con una letra mayúscula seguida de una o más letras minúsculas
"^[0123456789]{5}\$"	Busca coincidencias con exactamente cinco dígitos

Las expresiones regulares pueden ser muy útiles, pero si son demasiado largas podrían llegar a ser bastante ilegibles, y además exigirían un exhaustivo proceso de prueba.

6.11. COLECCIONES

Las colecciones son estructuras de datos que nos permiten guardar en su interior cualquier tipo de información. Existen diferentes tipos de colecciones y la forma en la que se accede, se guarda y se elimina la información es distinta en cada una de ellas.

Una de las grandes ventajas que presentan las colecciones es que permiten agregar elementos de forma dinámica. Es decir, es posible añadir o borrar elementos mientras se ejecuta el programa. En los arrays que ya vimos en apartados anteriores, esto no se podía hacer, sino que teníamos que indicar la

cantidad de elementos que el array iba a tener antes de comenzar a trabajar con él.

6.11.1. Lists

Una **lista** es una colección que nos permite guardar elementos del mismo tipo, de la misma forma que hace un array. Al contrario que en los arrays, las listas no tienen un tamaño definido, sino que van creciendo a medida que se les añade elementos nuevos.

Para poder usar una lista será necesario especificar arriba en nuestro programa que vamos a usar el espacio de nombres: `System.Collections.Generic`.

Para definir una lista, usaremos la sintaxis:

```
List<<tipoDatos>> <nombreLista>;
```

```
Ej.: List lista;
```

Además de definirla, deberemos crearla para poderla usar, lo que haremos con:

```
<nombreLista> = new List<<tipoDatos>>();
```

Normalmente haremos ambas cosas en una sola línea.

EJEMPLOS:

```
//Colección de enteros
List<int> listaEnteros = new List<int>();
//Colección heterogénea de objetos
List<Object> listaHeterogenea = new List<Object>();
```

Las listas se usan de manera similar a los arrays, con unas cuantas diferencias:

- ❖ Las listas *no tienen un tamaño definido*, por lo que a la hora de crearlas no hay que especificar el tamaño.
- ❖ Para *añadir* un nuevo dato a la lista, usaremos la función **Add()**, que nos añade un elemento en la última posición. Así, iremos añadiendo todos los valores que necesitemos.

```
lista.Add(25);
```

En un array, teníamos que primero reservar el tamaño que necesitáramos y luego ir modificando los valores de cada posición.

- ❖ Para saber cuántos elementos hay en una lista, usaremos la propiedad **Count**. En un array usábamos **Length** que nos devolvía el tamaño que habíamos reservado.

```
Console.WriteLine(lista.Count);
```

Salvo estas diferencias, una lista se puede usar básicamente de la misma forma que se usa un array. No obstante, las listas disponen de funciones específicas que nos permiten realizar las operaciones más comunes. A continuación, se enumeran las más importantes:

- ❖ **void Add(valor)**: Añade un valor a la lista en la última posición. El valor tiene que ser del mismo tipo del que hayamos definido la lista (esto es, en una lista de enteros sólo podemos almacenar enteros).
- ❖ **void Insert(int indice, valor)**: Inserta un valor en la lista en la posición especificada. La posición 0 se corresponde con el inicio de la lista (se introduciría el elemento al principio), mientras que la posición lista.Count -1 se corresponde a la última posición (similar a usar Add).
- ❖ **int IndexOf(valor)**: Nos devuelve la posición del elemento, si existe. Si el elemento no existe, nos devolverá -1.
- ❖ **bool Contains(valor)**: Nos devuelve true si el valor existe dentro de la lista y false en caso contrario.
- ❖ **void Clear()**: Elimina todos los elementos de la lista y nos la deja preparada para volver a introducir datos.
- ❖ **void RemoveAt(int posicion)**: Elimina el elemento que se encuentra en la posición especificada. Recordemos que la primera posición es la 0 y la última será la Count-1.
- ❖ **bool Remove(valor)**: Elimina la primera vez que aparece el valor dentro de la lista. Nos devolverá true si ha encontrado y borrado el valor y false si el valor no se ha encontrado dentro de la lista.
- ❖ **int RemoveAll(valor)**: Elimina todas las veces que aparece el valor dentro de la lista. Nos devolverá un entero que se corresponderá al número de elementos que han sido eliminados.
- ❖ **void AddRange(colección)**: Copia dentro de la lista todos los elementos de la colección. Esta colección puede ser otra lista, un array, etc.
- ❖ **public void TrimExcess()**: Establece la capacidad de la lista (el nº de elementos que puede almacenar) en el número real de elementos, para así no desperdiciar memoria.
- ❖ Otras funciones interesantes incluidas son: **Average** (media aritmética), **Max**, **Min**, **Reverse** (invierte el orden de los elementos), **Sort** (ordena) y **Sum**.

EJEMPLO:

```
static void Main(string[] args)
{
    List<string> equipos = new List<string>();
}
```

```
Console.WriteLine("CAPACIDAD al crear la lista: {0}", equipos.Capacity);

//Añado 4 elementos
equipos.Add("Real Madrid");
equipos.Add("Atlético Madrid");
equipos.Add("Real Sociedad");
equipos.Add("Sevilla");

//Muestro la lista
Console.WriteLine("\n-----LISTA INICIAL:");
MostrarLista(equipos);

//Compruebo si está el Sevilla y en caso afirmativo, devuelvo su posición
if(equipos.Contains("Sevilla"))
    Console.WriteLine("\nEl equipo \"Sevilla\" está en la posición
{0}", equipos.IndexOf("Sevilla"));

//Inserto Osasuna en la posición 2
equipos.Insert(2, "Osasuna");

//Muestro la lista
Console.WriteLine("\n-----LISTA después de Insert(2,\"Osasuna\"):");
MostrarLista(equipos);

//Borro Osasuna y al primer elemento
equipos.Remove("Osasuna");
equipos.RemoveAt(0);
//Añado una nueva lista
equipos.AddRange(new[] { "Rayo Vallecano", "Barcelona" });

//Muestro la lista
Console.WriteLine("\n-----LISTA después de Remove(\"Osasuna\"),
RemoveAt(0) y de añadir la nueva lista:");
MostrarLista(equipos);

//Ordeno la lista
equipos.Sort();
//Muestro la lista
Console.WriteLine("\n-----LISTA después de Sort:");
MostrarLista(equipos);

//Reasigno la capacidad eliminando lo innecesario
UpdateCapacidad(equipos);

//Borro todos los elementos de la lista
equipos.Clear();
//Muestro la lista
Console.WriteLine("\n-----ACTUALIZACIÓN de nº de elementos después de
Clear");
MostrarCapacidadYCount(equipos);

//Reasigno la capacidad eliminando lo innecesario
UpdateCapacidad(equipos);

Console.ReadLine();
}
```

```
//Procedimiento para mostrar la lista
static void MostrarLista(List<string> equipos)
{
    int i = 0;
    foreach (var equipo in equipos)
    {
        Console.WriteLine("equipos[{0}]=1,-20", i, equipo);
        i++;
        if ((i % 2 == 0 && i != 0) || i == equipos.Count)
            Console.WriteLine("\n");
    }
    MostrarCapacidadYCount(equipos);
}

//Procedimiento para mostrar la capacidad y el nº de elementos
static void MostrarCapacidadYCount(List<string> equipos) =>
    Console.WriteLine($"-----CAPACIDAD:{equipos.Capacity}, Nº DE ELEMENTOS:
{equipos.Count}");

//Procedimiento para actualizar la capacidad y mostrarla
static void UpdateCapacidad(List<string> equipos)
{
    equipos.TrimExcess();
    Console.WriteLine("\n-----ACTUALIZACIÓN de capacidad después de
TrimExcess");
    MostrarCapacidadYCount(equipos);
}
```

6.11.1.1. CUÁNDO USAR LISTS Y ARRAYS

Declarar y añadir elementos a una matriz de tipo `int` es más eficiente que hacerlo a una de tipo `List<int>`.

En términos generales, una matriz o array es recomendable cuando sabemos el número fijo de elementos que va a haber en la colección o el número máximo de elementos posibles. No es recomendable sin embargo, cuando el número de elementos máximo es desconocido o simplemente no lo sabemos, pudiendo incrementarse o no según la ejecución de los procesos.

`List<T>` nos proporciona ventajas a la hora de acceder a los elementos de una colección, y aunque es menos práctica de cara al rendimiento cuando tenemos claro el número máximo de elementos, podría seguir siendo interesante su uso en esas circunstancias si las operaciones que tenemos que hacer con los elementos de la colección son costosas. No obstante, en el caso de desconocer el número máximo o fijo de elementos, es mejor que array.

No podemos por lo tanto, afirmar que uno u otro es mejor que el otro. Qué utilizar en cada momento dependerá de nuestras necesidades.

6.11.2. HashTables

Una tabla hash es una estructura de datos algo compleja. C# nos provee de una clase que nos da toda la funcionalidad necesaria y podemos utilizarla tan fácilmente como las demás colecciones.

Una tabla hash es una colección indexada, es decir, vamos a tener un índice y un valor referenciado a ese índice. Sin embargo, la indexación no se lleva a cabo como en las listas. En su lugar, el **hashtable** coloca el elemento asociado a un valor conocido como *key*, *llave* o *clave*. Es como un diccionario en el que se van almacenando datos como pares de *clave* y *valor*.

Para poder usar una tabla hash será necesario especificar arriba en nuestro programa que vamos a usar el espacio de nombres: `System.Collections`.

Para definir una hashtable, usaremos la sintaxis:

```
Hashtable <nombreHashTable>;
```

```
Ej.: Hashtable tablahash;
```

Además de definirla, deberemos crearla para poderla usar, lo que haremos con:

```
<nombreHashTable> = new Hashtable();
```

Normalmente haremos ambas cosas en una sola línea.

EJEMPLO:

```
Hashtable miHashtable = new Hashtable();
```

Para saber cuántos elementos hay en un hashtable, al igual que con las listas, usaremos la propiedad **Count**.

Además, esta colección nos ofrece una serie de métodos, entre los que destacamos:

- ❖ **void Add(key, valor):** Añade un elemento con la clave y el valor especificados. El valor puede ser *null*.
- ❖ **bool ContainsKey(key):** Nos devuelve true si la clave existe dentro del hashtable y false en caso contrario.
- ❖ **bool ContainsValue(valor):** Nos devuelve true si el valor existe dentro del hashtable y false en caso contrario.
- ❖ **void Clear():** Elimina todos los elementos del hashtable y nos lo deja preparado para volver a introducir datos.
- ❖ **void Remove(key):** Elimina el elemento cuya clave coincide con la introducida como parámetro.

EJEMPLO:

```
public static void Main()
{
    // Creo un nuevo hashtable
    Hashtable abrirCon = new Hashtable();

    // Añado algunos elementos
    // No puede haber claves duplicadas, pero los valores sí pueden estarlo
    abrirCon.Add("txt", "notepad.exe");
    abrirCon.Add("bmp", "paint.exe");
    abrirCon.Add("dib", "paint.exe");
    abrirCon.Add("rtf", "wordpad.exe");

    // Si trato de añadir una clave existente se produce una excepción
    try
    {
        abrirCon.Add("txt", "winword.exe");
    }
    catch
    {
        Console.WriteLine("EXCEPCIÓN:");
        Console.WriteLine("La Key = \"txt\" ya existe");
    }

    RecorrerHashTable(abrirCon);

    //Puedo cambiar el valor asociado a una clave
    abrirCon["rtf"] = "winword.exe";
    Console.WriteLine("\nCAMBIO EL VALOR ASOCIADO a \"rtf\"");
    Console.WriteLine("Para key = \"rtf\", valor = {0}.", abrirCon["rtf"]);

    //Si una clave no existe y se le asigna un valor
    //se añade al hashtable
    Console.WriteLine("\nLE ASIGNO UN VALOR A UNA CLAVE que no existía");
    Console.WriteLine("abrirCon[\"doc\"]=\"winword.exe\"");
    abrirCon["doc"] = "winword.exe";

    RecorrerHashTable(abrirCon);

    //Si no contiene una clave, se la añado
    if (!abrirCon.ContainsKey("ht"))
    {
        abrirCon.Add("ht", "hypertrm.exe");
        Console.WriteLine("\nAÑADO un nuevo par clave/valor:");
        Console.WriteLine("Valor añadido para la key = \"ht\": {0}",
abrirCon["ht"]);
    }

    RecorrerHashTable(abrirCon);

    // Recorro sólo las keys
    Console.WriteLine("\nRECORRIDO de las keys del hashtable");
    foreach (string key in abrirCon.Keys)
    {
        Console.WriteLine(String.Format("Key = {0}", key).PadRight(40));
    }
}
```

```
// Recorro sólo los valores
Console.WriteLine("\n\nRECORRIDO de los valores del hashtable");
foreach (string valor in abrirCon.Values)
{
    Console.Write(String.Format("Value = {0}", valor).PadRight(40));
}

//Borro un par clave/valor
Console.WriteLine("\n\nBORRO UN PAR CLAVE/VALOR:\nRemove(\"doc\")");
abrirCon.Remove("doc");

//Compruebo si está la clave "doc"
if (!abrirCon.ContainsKey("doc"))
{
    Console.WriteLine("CONTAINSKEY:\nKey \"doc\" no se ha encontrado.");
}

Console.WriteLine("El hashtable tiene {0} elementos", abrirCon.Count);

//Borro todos los elementos del hashtable
abrirCon.Clear();
Console.WriteLine("\nBORRO todo:\nEl hashtable tiene {0} elementos",
abrirCon.Count);

Console.ReadLine();
}

//Procedimiento para mostrar los elementos del hashtable
public static void RecorrerHashTable(Hashtable abrirCon)
{
    //Recorro el hashtable. Cada elemento va a ser un DictionaryEntry
    Console.WriteLine("\nRECORRIDO del hashtable:");
    foreach (DictionaryEntry par in abrirCon)
    {
        //Añado espacios al final para que todos tengan la misma longitud
        Console.Write(String.Format("Key = {0}, Valor = {1}", par.Key,
par.Value).PadRight(40));
    }
    Console.Write("\n");
}
```